

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

TÀI LIỆU HƯỚNG DẪN THỰC HÀNH

IT1140 – TIN HỌC ĐẠI CƯƠNG

(LƯU HÀNH NỘI BỘ)

BẢN DÀNH CHO GIẢNG VIÊN

© 2025 – HUST – SOICT

MỤC LỤC

BÀI THỰC HÀNH SỐ 1: CÚ PHÁP CƠ BẢN, BIỂU THỨC VÀ CÁC LUỒNG ĐIỀU KHIỂN 5

Exercise 1.1.1: Đổi nhiệt độ 7

Exercise 1.1.2: Tính tiền điện đơn giản 8

Exercise 1.1.3: Đổi thời gian 8

Exercise 1.2.1: Kiểm tra chẵn lẻ 9

Exercise 1.2.2: Số lớn nhất trong 3 số 9

Exercise 1.2.3: Xếp loại điểm 10

Exercise 1.2.4: Kiểm tra năm nhuận 11

Exercise 1.3.1: Tổng từ 1 đến n 11

Exercise 1.3.2: Đếm số chẵn trong đoạn 12

Exercise 1.3.3: Kiểm tra số nguyên tố 12

Bài 1.4: Phân loại chỉ số BMI 13

Bài 1.5: Tiền điện theo bậc thang 13

Bài 1.6: Giải phương trình bậc nhất 14

Bài 1.7: Tổng, trung bình, lớn nhất, nhỏ nhất 15

Bài 1.8: Bảng cửu chương 16

Bài 1.9: Số ngày trong tháng 17

Bài 1.10: Đếm chữ số và tính tổng các chữ số 17

Bài 1.11: Dãy Fibonacci 18

Bài 1.12: Ước chung lớn nhất và bội chung nhỏ nhất 18

Bài 1.13: Vẽ tam giác sao 19

BÀI THỰC HÀNH SỐ 2: HÀM VÀ CHUỖI 19

Bài 2.1: Hàm tính chu vi và diện tích hình chữ nhật 21

Bài 2.2: Hàm kiểm tra số nguyên tố 22

Bài 2.3: Rút gọn phân số 22

Bài 2.4: Máy tính bỏ túi 23

Bài 2.5: Chuẩn hóa họ tên 24

Bài 2.6: Thống kê từ trong câu 24
Bài 2.7: Nút gốc chữ số 25
Bài 2.8: Đếm số chẵn bằng hàm 25
Bài 2.9: Hàm tính lũy thừa 26
Bài 2.10: Đếm số câu 27
Bài 2.11: Phân loại chuỗi 27
Bài 2.12: Đếm số lần xuất hiện chuỗi con 28
BÀI THỰC HÀNH SỐ 3: CHUỖI (TIẾP), TUPLE, LIST, SET, DICTIONARY 28
Bài 3.1: Đổi sang hệ nhị phân (Chuỗi) 29
Bài 3.2: Bảng điểm lớp học (Tuple) 30
Bài 3.3: Kiểm tra mật khẩu mạnh (List) 31
Bài 3.4: Loại bỏ phần tử trùng lặp (Set) 32
Bài 3.5: Sổ điểm có cập nhật (Dictionary) 32
Bài 3.6: Kiểm tra chuỗi đối xứng 33
Bài 3.7: Đếm nguyên âm và phụ âm (chuỗi) 34
Bài 3.8: Thống kê trong một câu 34
Bài 3.9: Kiểm tra tính đơn điệu 35
Bài 3.10: Trộn hai dãy đã sắp xếp 36
Bài 3.11: Tính trung bình sau khi loại đỉnh 37
Bài 3.12: Xử lý danh sách số (LIST) 37
Bài 3.13: Tìm kiếm vị trí xuất hiện 38
BÀI THỰC HÀNH SỐ 4: MODULE VÀ TẬP TIN 39
Bài 4.1: Thống kê cơ bản một tệp văn bản 40
Bài 4.2: Ghi danh sách vào tệp 41
Bài 4.3: Tính toán trên tệp số 42
Bài 4.4: Tìm dòng chứa từ khóa 43
Bài 4.5: Đếm tần suất từ 44
Bài 4.6: Chuẩn hóa văn bản 45
Bài 4.7: Gộp 2 tệp 46

Bài 4.8: Bảng điểm dạng CSV 47
Bài 4.9: Lọc và ghi báo cáo 47
Bài 4.10: Nhật ký nhiệt độ 48
BÀI THỰC HÀNH SỐ 5: ĐỒ HỌA VỚI PYTHON **50**
Bài 5.1: Đa giác đều n cạnh 51
Bài 5.2: Đồ thị gấp khúc 51
Bài 5.3: Các hình cơ bản 52
Bài 5.4: Đường bậc thang 53
Bài 5.5: Hình xoắn ốc vuông 53
Bài 5.6: Hoa văn nhiều màu 54
Bài 5.7: Lưới ô vuông 54
Bài 5.8: Biểu đồ cột 55
Bài 5.9: Biểu đồ hình quạt 55
Bài 5.10: Biểu đồ điểm rải 56
Bài 5.11: Đồ thị hàm số 56
Bài 5.12: Đồ thị sin và cos 57
Bài 5.13: Biểu đồ tần suất 58
Bài 5.14: Kết hợp nhiều biểu đồ con 58
Bài 5.15: Đường cong tham số 59

BÀI THỰC HÀNH SỐ 1: CÚ PHÁP CƠ BẢN, BIỂU THỨC VÀ CÁC LUỒNG ĐIỀU KHIỂN

Tóm tắt kiến thức

Mục tiêu: Sau buổi thực hành, sinh viên mô tả được lời giải bằng thuật toán, viết chương trình Python đơn giản, và sử dụng thành thạo rẽ nhánh cùng vòng lặp để giải các bài toán cơ bản.

1. Thuật toán và biểu diễn thuật toán

Thuật toán là một dãy hữu hạn các bước rõ ràng nhằm giải quyết một bài toán: có đầu vào và đầu ra xác định, các bước không mơ hồ, kết thúc sau hữu hạn bước và cho kết quả đúng với dữ liệu hợp lệ. Có thể biểu diễn thuật toán bằng ngôn ngữ tự nhiên, giả mã hoặc lưu đồ. Ba cấu trúc điều khiển nền tảng là **tuần tự, rẽ nhánh và lặp**.

2. Cấu trúc chương trình và biến

Các câu lệnh Python thực hiện từ trên xuống dưới. Python phân biệt chữ hoa và chữ thường; `tong`, `Tong`, `TONG` là ba biến khác nhau. Nên đặt tên biến gợi nghĩa, dùng chữ thường và dấu gạch dưới: `diem_trung_binh`, `so_luong`.

Các kiểu dữ liệu cơ bản và phép toán:

Kiểu	Ý nghĩa	Ví dụ
`int`	Số nguyên	`10, -5, 0`
`float`	Số thực	`3.14, -2.5`
`str`	Chuỗi ký tự	`"Python"`
`bool`	Đúng/sai	`True, False`

Toán tử	+	-	*	/	//	%	
Ý nghĩa	cộng	trừ	nhân	chia thực	chia nguyên	chia dư	lũy thừa

3. Nhập và xuất dữ liệu

Xuất bằng `print()`. In một hoặc nhiều giá trị; các giá trị cách nhau một dấu cách. Có thể đổi ký tự ngăn cách bằng `sep` và ký tự kết thúc bằng `end`.

```
print("Tong la", 5 + 3)        # Tong la 8
print(1, 2, 3, sep="-")        # 1-2-3
print("A", end=" ")            # không xuống dòng
print("B")                     # A B
```

Nhập bằng `input()`. Đọc một dòng người dùng gõ vào và **luôn trả về kiểu chuỗi**, kể cả khi gõ số. Muốn tính toán phải ép kiểu bằng `int()` hoặc `float()`.

```
ten = input()                  # giữ nguyên chuỗi
n = int(input())               # ép sang số nguyên
x = float(input())             # ép sang số thực
```

Đây là lỗi kinh điển của người mới: `input() + 1` báo lỗi vì đang cộng chuỗi với số; phải viết `int(input()) + 1`.

Nhập nhiều số trên một dòng với `map`. Khi các số nằm trên cùng một dòng, làm hai bước: `split()` tách dòng thành danh sách các chuỗi, rồi `map(int, ...)` ép từng phần tử sang số.

```
a, b = map(int, input().split())      # "3 5"  -> a = 3, b = 5
so = list(map(int, input().split()))  # "1 2 3 4" -> [1, 2, 3, 4]
```

`map(ham, dãy)` áp dụng `ham` lên **mỗi** phần tử của dãy. `map(int, ...)` biến từng chuỗi thành `int`, `map(float, ...)` thành số thực. Kết quả của `map` cần được đưa vào `list(...)` khi muốn một danh sách, hoặc giải nén trực tiếp vào các biến như `a, b = ...` (số biến bên trái phải bằng số giá trị nhập vào).

Định dạng số thực. Chuỗi f (viết chữ `f` trước dấu nháy) cho phép chèn giá trị vào giữa chuỗi; phần `:.2f` làm tròn về 2 chữ số thập phân.

```
diem = 8.66667
print(f"{diem:.2f}")          # 8.67
print(f"Chao {ten}")          # chèn biến vào chuỗi
```

4. Biểu thức Boolean và rẽ nhánh

Biểu thức Boolean chỉ nhận `True`/`False`, thường xuất hiện trong điều kiện. Toán tử so sánh: `==` `!=` `<` `>` `<=` `>=`; toán tử logic: `and`, `or`, `not`. Python dùng **thụt lề** để xác định khối lệnh.

```
diem = float(input())
if diem >= 9:
    print("XUAT_SAC")
elif diem >= 6.5:
    print("KHA")
else:
    print("CHUA_DAT")
```

Python xét các điều kiện lần lượt từ trên xuống, chỉ chạy khối của điều kiện đúng đầu tiên, nên thứ tự các nhánh phải hợp lý.

5. Vòng lặp for và biến tích lũy

Dùng `for` khi biết trước số lần lặp. Nhớ `range(1, n + 1)` mới duyệt tới n vì giá trị cuối không được lấy.

```
tong = 0
for i in range(1, n + 1):
    tong += i
```

Biến tích lũy (khởi tạo trước, cập nhật dần trong vòng lặp) là mẫu chung cho tính tổng, đếm, tìm lớn nhất/nhỏ nhất. Khi tìm max/min hãy khởi tạo bằng phần tử đầu tiên, không khởi tạo bằng 0.

6. Vòng lặp while, break và continue

Dùng `while` khi số lần lặp phụ thuộc điều kiện. Vòng `while` đúng cần: khởi tạo biến điều khiển, điều kiện dừng, và câu lệnh làm thay đổi biến điều khiển trong thân vòng. `break` thoát vòng lặp ngay; `continue` bỏ qua phần còn lại của lần lặp hiện tại.

```
n = int(input())
while n < 0:
    n = int(input())     # thiếu dòng này sẽ lặp vô hạn
```

Vòng lặp có thể **lồng nhau** (vòng ngoài ứng với dòng, vòng trong ứng với cột). Khi cần biết vòng lặp kết thúc vì lý do gì, dùng thêm một **biến cờ** Boolean.

7. Lưu ý cốt lõi khi viết code

Thụt đầu dòng (indentation). Python dùng khoảng trắng đầu dòng để xác định khối lệnh — đây là cú pháp bắt buộc, không phải để cho đẹp. Sau mỗi dòng kết thúc bằng dấu hai chấm (`if`, `else`, `for`, `while`, `def`), các lệnh thuộc khối đó phải thụt vào; quy ước là **4 dấu cách** cho mỗi mức.

```
n = int(input())
```

```
if n > 0:
    print("Duong")        # thuộc khối if (thụt 4 dấu cách)
    print("Da xu ly")     # cùng khối, cùng mức thụt
print("Ket thuc")         # ngoài khối if (không thụt)
```

- Các lệnh trong **cùng một khối** phải thụt **cùng một mức**; lệch mức sẽ gây lỗi `IndentationError`.
- **Không trộn tab và dấu cách** trong cùng một tệp — dễ gây lỗi khó thấy. Trình soạn thảo (PyCharm, Thonny) tự chèn dấu cách khi bạn gõ Enter sau dấu hai chấm.
- Thụt lề sai làm một câu lệnh nằm nhầm trong hoặc ngoài vòng lặp, khiến chương trình chạy sai dù không báo lỗi.

Khai báo và sử dụng thư viện. Khi cần một hàm không có sẵn (căn bậc hai, số ngẫu nhiên, vẽ đồ thị…), phải nạp thư viện bằng lệnh `import`, và **đặt các lệnh `import` ở đầu tệp**.

```
import math                     # nạp cả thư viện, gọi qua tên đầy đủ
print(math.sqrt(16))            # 4.0
print(math.pi)                  # 3.14159...

from math import sqrt, pi       # chỉ lấy một số tên cụ thể
print(sqrt(16))                 # gọi trực tiếp, không cần math.
```

- `import math` rồi gọi `math.sqrt(...)`; hoặc `from math import sqrt` rồi gọi thẳng `sqrt(...)`.
- Thư viện chuẩn (`math`, `random`, `statistics`…) dùng được ngay; gói bên thứ ba (`matplotlib`…) phải cài thêm bằng `pip`.
- Quên `import` sẽ gây lỗi `NameError` (tên chưa được định nghĩa).

Một vài quy ước giúp mã dễ đọc, ít lỗi. Đặt tên biến gợi nghĩa, dùng chữ thường và dấu gạch dưới (`diem_trung_binh`). Dùng dấu `#` để ghi chú thích cho người đọc. Mỗi câu lệnh viết trên một dòng riêng.

Lưu ý và lỗi thường gặp

- Quên ép kiểu sau `input()` nên đem cộng chuỗi thay vì số.
- Nhầm `=` (gán) với `==` (so sánh) trong điều kiện.
- Thứ tự các nhánh `elif` không hợp lý khiến một nhánh không bao giờ được chọn.
- Sai giới hạn `range` (thừa/thiếu một lần lặp).
- Quên khởi tạo biến tích lũy hoặc khởi tạo sai giá trị ban đầu.
- Đặt lệnh cập nhật biến điều khiển ngoài thân `while` gây lặp vô hạn.
- Sai thụt lề hoặc trộn tab với dấu cách (`IndentationError`); dùng nhất quán 4 dấu cách.
- Quên `import` thư viện trước khi dùng, gây `NameError`.

Ghi nhớ trước khi nộp bài

Chạy thử với dữ liệu thường, giá trị biên (0, 1, số âm, các mốc chuyển bậc) và trường hợp đặc biệt; bảo đảm chương trình chỉ in đúng phần kết quả, không in thêm lời nhắc; kiểm tra số dòng và thứ tự dòng của phần xuất.

Thực hành trên lớp

Bài 1.1: Làm quen với cú pháp và các biểu thức

Exercise 1.1.1: Đổi nhiệt độ

Chủ đề: Nhập xuất, phép toán

Nhập nhiệt độ Celsius `C` (số thực). In nhiệt độ Fahrenheit theo công thức `F = C × 9 / 5 + 32`, làm tròn 2 chữ số thập phân.

Input: Một số thực `C`.
Output: Giá trị `F` với 2 chữ số thập phân.
Code mẫu:

```
c = float(input())
f = c * 9 / 5 + 32
print(f"{f:.2f}")
```

Test case:

#	Input	Output
1	0	32.00
2	100	212.00
3	37	98.60
4	-40	-40.00
5	25.5	77.90

Hướng dẫn: Đọc số thực bằng float(input()), áp công thức F = C×9/5+32 và in với 2 chữ số thập phân bằng f-string.

Exercise 1.1.2: Tính tiền điện đơn giản

Chủ đề: Nhập xuất, phép toán

Nhập số điện tiêu thụ `n` (kWh) và đơn giá `p` (đồng/kWh). Tính tổng tiền điện.
Input: Hai số nguyên `n`, `p`, cách nhau bởi dấu cách.
Output: Tổng tiền điện.
Code mẫu:

```
n, p = map(int, input().split())
print(n * p)
```

Test case:

#	Input	Output
1	100 2000	200000
2	0 3000	0
3	50 1800	90000
4	123 2500	307500
5	999 1500	1498500

Hướng dẫn: Đọc hai số nguyên trên một dòng bằng map(int, input().split()), rồi nhân số điện với đơn giá.

Exercise 1.1.3: Đổi thời gian

Chủ đề: Phép chia nguyên và dư

Nhập số nguyên `n` là tổng số giây. Đổi thành giờ, phút, giây.
Input: Một số nguyên không âm `n`.
Output: Ba số nguyên `gio phut giay`, cách nhau bởi dấu cách.
Code mẫu:

```
n = int(input())

gio = n // 3600
```

```
n %= 3600
phut = n // 60
giay = n % 60

print(gio, phut, giay)
```

Test case:

#	Input	Output
1	0	0 0 0
2	59	0 0 59
3	60	0 1 0
4	3672	1 1 12
5	86400	24 0 0

Hướng dẫn: Dùng // để lấy giờ, % để lấy phần dư, sau đó tách tiếp phút và giây từ phần dư.

Bài 1.2: Cấu trúc điều khiển rẽ nhánh

Exercise 1.2.1: Kiểm tra chẵn lẻ

Chủ đề: Rẽ nhánh, phép %

Nhập số nguyên n. In CHAN nếu n là số chẵn, ngược lại in LE.

Input: Một số nguyên n.

Output: CHAN hoặc LE.

Code mẫu:

```
n = int(input())

if n % 2 == 0:
    print("CHAN")
else:
    print("LE")
```

Test case:

#	Input	Output
1	18	CHAN
2	1	LE
3	0	CHAN
4	-7	LE
5	-20	CHAN

Hướng dẫn: Kiểm tra n % 2 == 0 để phân biệt số chẵn và số lẻ.

Exercise 1.2.2: Số lớn nhất trong 3 số

Chủ đề: Rẽ nhánh

Nhập ba số nguyên a, b, c. In số lớn nhất.

Input: Ba số nguyên, cách nhau bởi dấu cách.

Output: Giá trị lớn nhất.

Code mẫu:

```
a, b, c = map(int, input().split())
```

```
max_value = a
if b > max_value:
    max_value = b
if c > max_value:
    max_value = c

print(max_value)
```

Test case:

#	Input	Output
1	1 2 3	3
2	10 5 7	10
3	4 4 2	4
4	-1 -5 -3	-1
5	0 0 0	0

Hướng dẫn: Giả sử số đầu là lớn nhất, so sánh lần lượt với hai số còn lại và cập nhật khi gặp số lớn hơn.

Exercise 1.2.3: Xếp loại điểm

Chủ đề: Rẽ nhánh nhiều nhánh

Nhập điểm tổng kết `d` trong khoảng từ 0 đến 10. In xếp loại theo bảng:

- XUAT_SAC nếu d >= 9
- GIOI nếu 8
- KHA nếu 6.5
- TRUNG_BINH nếu 5
- KHONG_DAT nếu d

Input: Một số thực `d`.

Output: Xếp loại tương ứng.

Code mẫu:

```
d = float(input())

if d >= 9:
    print("XUAT_SAC")
elif d >= 8:
    print("GIOI")
elif d >= 6.5:
    print("KHA")
elif d >= 5:
    print("TRUNG_BINH")
else:
    print("KHONG_DAT")
```

Test case:

#	Input	Output
1	9.5	XUAT_SAC
2	8	GIOI
3	7.25	KHA
4	5	TRUNG_BINH
5	4.99	KHONG_DAT

Hướng dẫn: Chuỗi if–elif xếp theo thứ tự giảm dần của mốc điểm; nhánh đúng đầu tiên quyết định xếp loại.

Exercise 1.2.4: Kiểm tra năm nhuận

Chủ đề: Rẽ nhánh, toán tử logic

Nhập năm `y`. Một năm là năm nhuận nếu chia hết cho 400, hoặc chia hết cho 4 nhưng không chia hết cho 100.

Input: Một số nguyên dương `y`.

Output: `NAM_NHUAN` hoặc `KHONG_NHUAN`

Code mẫu:

```
y = int(input())

if y % 400 == 0 or (y % 4 == 0 and y % 100 != 0):
    print("NAM_NHUAN")
else:
    print("KHONG_NHUAN")
```

Test case:

#	Input	Output
1	2024	NAM_NHUAN
2	2025	KHONG_NHUAN
3	2000	NAM_NHUAN
4	1900	KHONG_NHUAN
5	2100	KHONG_NHUAN

Hướng dẫn: Năm nhuận khi chia hết 400, hoặc chia hết 4 mà không chia hết 100; ghép điều kiện bằng and/or.

Bài 1.3: Vòng lặp cơ bản

Exercise 1.3.1: Tổng từ 1 đến n

Chủ đề: Vòng lặp, biến tích lũy

Nhập số nguyên dương `n`. Dùng vòng lặp để tính tổng `S = 1 + 2 + ... + n`.

Input: Một số nguyên dương `n`.

Output: Giá trị tổng `S`.

Code mẫu:

```
n = int(input())

tong = 0
for i in range(1, n + 1):
    tong += i

print(tong)
```

Test case:

#	Input	Output
1	1	1
2	2	3
3	10	55
4	100	5050

5	1000	500500

Hướng dẫn: Khởi tạo tong = 0 rồi cộng dồn i qua range(1, n+1).

Exercise 1.3.2: Đếm số chẵn trong đoạn

Chủ đề: Vòng lặp, đếm có điều kiện

Nhập hai số nguyên `a`, `b` với `a <= b`. Đếm số lượng số chẵn thuộc đoạn đóng `[a, b]`.

Input: Hai số nguyên `a`, `b`, cách nhau bởi dấu cách.

Output: Số lượng số chẵn.

Code mẫu:

```
a, b = map(int, input().split())

dem = 0
for i in range(a, b + 1):
    if i % 2 == 0:
        dem += 1

print(dem)
```

Test case:

#	Input	Output
1	1 10	5
2	2 2	1
3	3 3	0
4	-4 4	5
5	10 20	6

Hướng dẫn: Duyệt từ a đến b, tăng biến đếm mỗi khi gặp số chẵn.

Exercise 1.3.3: Kiểm tra số nguyên tố

Chủ đề: Vòng lặp, cờ hiệu

Nhập số nguyên dương `n`. In `SO_NGUYEN_TO` nếu `n` là số nguyên tố; ngược lại in `KHONG_PHAI_SO_NGUYEN_TO`.

Số nguyên tố là số lớn hơn 1 và chỉ có hai ước dương là 1 và chính nó.

Input: Một số nguyên dương `n`.

Output: `SO_NGUYEN_TO` hoặc `KHONG_PHAI_SO_NGUYEN_TO`.

Code mẫu:

```
n = int(input())

la_nguyen_to = n >= 2

for i in range(2, n):
    if n % i == 0:
        la_nguyen_to = False
        break

if la_nguyen_to:
    print("SO_NGUYEN_TO")
else:
    print("KHONG_PHAI_SO_NGUYEN_TO")
```

Test case:

#	Input	Output
1	1	KHONG_PHAI_SO_NGUYEN_TO
2	2	SO_NGUYEN_TO
3	17	SO_NGUYEN_TO
4	100	KHONG_PHAI_SO_NGUYEN_TO
5	97	SO_NGUYEN_TO

Hướng dẫn: Dùng biến cờ Boolean; thử chia n cho các số nhỏ hơn, nếu chia hết thì không phải số nguyên tố.

Bài 1.4: Phân loại chỉ số BMI

Chủ đề: Rẽ nhánh nhiều nhánh

Nhập cân nặng m (kg) và chiều cao h (m). Tính BMI = m / h^2. In BMI với 2 chữ số thập phân, sau đó xuống dòng in phân loại: THIEU_CAN nếu BMI < 18.5; BINH_THUONG nếu 18.5 ≤ BMI < 25; THUA_CAN nếu 25 ≤ BMI < 30; BEO_PHI nếu BMI ≥ 30.

Input: Hai số thực m và h, cách nhau bởi dấu cách.

Output: Dòng 1: giá trị BMI với 2 chữ số thập phân. Dòng 2: tên phân loại.

Code mẫu:

```
m, h = map(float, input().split())
bmi = m / (h * h)
print(f"{bmi:.2f}")
if bmi < 18.5:
    print("THIEU_CAN")
elif bmi < 25:
    print("BINH_THUONG")
elif bmi < 30:
    print("THUA_CAN")
else:
    print("BEO_PHI")
```

Test case:

#	Input	Output
1	50 1.7	17.30 THIEU_CAN
2	68 1.7	23.53 BINH_THUONG
3	80 1.7	27.68 THUA_CAN
4	95 1.7	32.87 BEO_PHI
5	53.4 1.7	18.48 THIEU_CAN

Hướng dẫn: Tính BMI = m/h^2 rồi phân loại bằng chuỗi if–elif theo các mốc 18.5/25/30.

Bài tập tự luyện (không bắt buộc)

Bài 1.5: Tiền điện theo bậc thang

Chủ đề: Rẽ nhánh theo khoảng

Nhập số điện tiêu thụ n (kWh, số nguyên không âm). Tính tiền điện theo ba bậc: 50 kWh đầu tiên giá 1800 đồng/kWh; 50 kWh tiếp theo giá 2000 đồng/kWh; từ kWh thứ 101 trở đi giá 2500 đồng/kWh. In tổng tiền (số nguyên).

Input: Một số nguyên không âm n.

Output: Tổng tiền điện.

Code mẫu:

```
n = int(input())
if n <= 50:
    tien = n * 1800
elif n <= 100:
    tien = 50 * 1800 + (n - 50) * 2000
else:
    tien = 50 * 1800 + 50 * 2000 + (n - 100) * 2500
print(tien)
```

Test case:

#	Input	Output
1	0	0
2	50	90000
3	51	92000
4	100	190000
5	250	565000

Hướng dẫn: Chia ba bậc, mỗi bậc tính phần điện thuộc bậc đó rồi cộng lại; điều kiện lồng theo mức tiêu thụ.

Bài 1.6: Giải phương trình bậc nhất

Chủ đề: Rẽ nhánh, trường hợp đặc biệt

Nhập hai số nguyên a và b. Giải phương trình ax + b = 0. Nếu a khác 0, in nghiệm x với 2 chữ số thập phân. Nếu a = 0 và b = 0, in VO_SO_NGHIEM. Nếu a = 0 và b khác 0, in VO_NGHIEM.

Input: Hai số nguyên a và b, cách nhau bởi dấu cách.

Output: Giá trị nghiệm với 2 chữ số thập phân, hoặc VO_SO_NGHIEM, hoặc VO_NGHIEM.

Code mẫu:

```
a, b = map(int, input().split())
if a != 0:
    x = -b / a
    print(f"{x:.2f}")
elif b == 0:
    print("VO_SO_NGHIEM")
else:
    print("VO_NGHIEM")
```

Test case:

#	Input	Output
1	2 -6	3.00
2	0 0	VO_SO_NGHIEM
3	0 5	VO_NGHIEM
4	-3 9	3.00
5	4 3	-0.75

Hướng dẫn: a khác 0 cho nghiệm −b/a; khi a = 0 tách hai trường hợp vô số nghiệm và vô nghiệm.

Bài 1.7: Tổng, trung bình, lớn nhất, nhỏ nhất

Chủ đề: Vòng lặp, nhiều biến tích lũy

Nhập số nguyên dương n, sau đó nhập n số nguyên trên cùng một dòng. In tổng, giá trị lớn nhất, giá trị nhỏ nhất và trung bình cộng (2 chữ số thập phân), mỗi giá trị trên một dòng. Không dùng các hàm sum, max, min có sẵn.

Input: Dòng 1: số nguyên dương n. Dòng 2: n số nguyên cách nhau bởi dấu cách.

Output: Bốn dòng: tổng, lớn nhất, nhỏ nhất, trung bình cộng (2 chữ số thập phân).

Code mẫu:

```
n = int(input())
so = input().split()
tong = 0
lon_nhat = int(so[0])
nho_nhat = int(so[0])
for i in range(n):
    x = int(so[i])
    tong += x
    if x > lon_nhat:
        lon_nhat = x
    if x < nho_nhat:
        nho_nhat = x
print(tong)
print(lon_nhat)
print(nho_nhat)
print(f"{tong / n:.2f}")
```

Test case:

#	Input	Output
1	1 7	7 7 7 7.00
2	5 1 2 3 4 5	15 5 1 3.00
3	4 -1 -2 -3 -4	-10 -1 -4 -2.50
4	3 10 10 10	30 10 10 10.00
5	6 5 -3 12 0 7 -8	13 12 -8 2.17

Hướng dẫn: Khởi tạo lớn nhất/nhỏ nhất bằng phần tử đầu, vừa cộng dồn tổng vừa cập nhật max/min trong một vòng lặp.

Bài 1.8: Bảng cửu chương

Chủ đề: Vòng lặp lồng nhau

Nhập hai số nguyên a và b với 1 ≤ a ≤ b ≤ 9. In bảng cửu chương của các số từ a đến b. Mỗi bảng gồm 10 dòng theo mẫu i x j = k, giữa hai bảng liên tiếp in một dòng trống.

Input: Hai số nguyên a và b, cách nhau bởi dấu cách.

Output: Các bảng cửu chương theo mô tả trên.

Code mẫu:

```
a, b = map(int, input().split())
for i in range(a, b + 1):
    if i > a:
        print()
    for j in range(1, 11):
        print(f"{i} x {j} = {i * j}")
```

Test case:

#	Input	Output
1	2 2	2 x 1 = 2 2 x 2 = 4 2 x 3 = 6 2 x 4 = 8 2 x 5 = 10 2 x 6 = 12 2 x 7 = 14 2 x 8 = 16 2 x 9 = 18 2 x 10 = 20
2	3 4	3 x 1 = 3 3 x 2 = 6 3 x 3 = 9 3 x 4 = 12 3 x 5 = 15 3 x 6 = 18 3 x 7 = 21 3 x 8 = 24 3 x 9 = 27 3 x 10 = 30 4 x 1 = 4 4 x 2 = 8 4 x 3 = 12 4 x 4 = 16 4 x 5 = 20 4 x 6 = 24 4 x 7 = 28 4 x 8 = 32 4 x 9 = 36 4 x 10 = 40
3	9 9	9 x 1 = 9 9 x 2 = 18 9 x 3 = 27 9 x 4 = 36 9 x 5 = 45 9 x 6 = 54 9 x 7 = 63 9 x 8 = 72

		9 x 9 = 81 9 x 10 = 90

Hướng dẫn: Vòng ngoài chạy từ a đến b, vòng trong in 10 dòng; in một dòng trống giữa các bảng.

Bài 1.9: Số ngày trong tháng

Chủ đề: Rẽ nhánh, toán tử in

Nhập tháng t và năm y. In số ngày của tháng đó. Tháng 4, 6, 9, 11 có 30 ngày; tháng 2 có 29 ngày nếu y là năm nhuận và 28 ngày nếu ngược lại; các tháng còn lại có 31 ngày. Nếu t không thuộc đoạn [1, 12], in KHONG_HOP_LE.

Input: Hai số nguyên t và y, cách nhau bởi dấu cách.

Output: Số ngày của tháng, hoặc KHONG_HOP_LE.

Code mẫu:

```
t, y = map(int, input().split())
if t < 1 or t > 12:
    print("KHONG_HOP_LE")
elif t in (4, 6, 9, 11):
    print(30)
elif t == 2:
    if y % 400 == 0 or (y % 4 == 0 and y % 100 != 0):
        print(29)
    else:
        print(28)
else:
    print(31)
```

Test case:

#	Input	Output
1	1 2025	31
2	2 2024	29
3	2 1900	28
4	11 2025	30
5	13 2025	KHONG_HOP_LE

Hướng dẫn: Dùng in để gom nhóm tháng 30 ngày; tháng 2 kiểm tra năm nhuận; các tháng còn lại 31 ngày.

Bài 1.10: Đếm chữ số và tính tổng các chữ số

Chủ đề: Vòng lặp while, tách chữ số

Nhập số nguyên n (có thể âm). Đếm số chữ số của n và tính tổng các chữ số (bỏ qua dấu âm). Quy ước số 0 có một chữ số và tổng chữ số bằng 0.

Input: Một số nguyên n.

Output: Hai số nguyên: số chữ số và tổng các chữ số, cách nhau bởi dấu cách.

Code mẫu:

```
n = int(input())
if n < 0:
    n = -n
if n == 0:
    print(1, 0)
else:
```

```
    dem = 0
    tong = 0
    while n > 0:
        tong += n % 10
        n //= 10
        dem += 1
    print(dem, tong)
```

Test case:

#	Input	Output
1	0	1 0
2	7	1 7
3	1234	4 10
4	-9081	4 18
5	1000000	7 1

Hướng dẫn: Dùng % 10 lấy chữ số cuối, // 10 để bỏ đi, lặp tới khi hết; số 0 xử lý riêng.

Bài 1.11: Dãy Fibonacci

Chủ đề: Vòng lặp, hoán đổi biến

Nhập số nguyên dương n. In n số đầu tiên của dãy Fibonacci trên một dòng, cách nhau bởi dấu cách. Dãy Fibonacci bắt đầu bằng 0 và 1; mỗi số tiếp theo bằng tổng hai số liền trước.

Input: Một số nguyên dương n.

Output: n số Fibonacci đầu tiên, cách nhau bởi dấu cách.

Code mẫu:

```
n = int(input())
a = 0
b = 1
ket_qua = ""
for i in range(n):
    ket_qua += str(a) + " "
    a, b = b, a + b
print(ket_qua.strip())
```

Test case:

#	Input	Output
1	1	0
2	2	0 1
3	5	0 1 1 2 3
4	10	0 1 1 2 3 5 8 13 21 34
5	15	0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Hướng dẫn: Giữ hai số liền trước, mỗi bước cập nhật a, b = b, a + b.

Bài 1.12: Ước chung lớn nhất và bội chung nhỏ nhất

Chủ đề: Thuật toán Euclid

Nhập hai số nguyên dương a và b. Dùng thuật toán Euclid để tính ước chung lớn nhất, sau đó tính bội chung nhỏ nhất theo công thức BCNN = a × b / UCLN.

Input: Hai số nguyên dương a và b, cách nhau bởi dấu cách.

Output: Hai số nguyên: UCLN và BCNN, cách nhau bởi dấu cách.

Code mẫu:

```
a, b = map(int, input().split())
x = a
y = b
while y != 0:
    x, y = y, x % y
print(x, a * b // x)
```

Test case:

#	Input	Output
1	12 18	6 36
2	7 13	1 91
3	100 25	25 100
4	1 999	1 999
5	270 192	6 8640

Hướng dẫn: Lặp x, y = y, x % y đến khi y = 0 được UCLN; BCNN = a×b // UCLN.

Bài 1.13: Vẽ tam giác sao

Chủ đề: Vòng lặp, nhân chuỗi

Nhập số nguyên dương n (n ≤ 20). In một tam giác cân bằng dấu sao có n dòng: dòng thứ i gồm n - i dấu cách rồi 2i - 1 dấu sao. Không có dấu cách thừa ở cuối mỗi dòng.

Input: Một số nguyên dương n.

Output: Tam giác sao gồm n dòng.

Code mẫu:

```
n = int(input())
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))
```

Test case:

#	Input	Output
1	1	*
2	3	 ***
3	5	 *********

Hướng dẫn: Dòng i gồm (n−i) dấu cách và (2i−1) dấu sao, tạo nhanh bằng phép nhân chuỗi.

BÀI THỰC HÀNH SỐ 2: HÀM VÀ CHUỖI

Tóm tắt kiến thức

Mục tiêu: Sau buổi thực hành, sinh viên viết được hàm có tham số và giá trị trả về, và xử lý chuỗi ký

tự bằng các phương thức thông dụng.

1. Vì sao cần hàm

Hàm là một khối lệnh có tên, nhận dữ liệu đầu vào và trả về kết quả. Chia chương trình thành hàm giúp tránh viết lặp, đặt tên cho từng bước xử lý và kiểm thử độc lập.

```
def dien_tich_hinh_tron(ban_kinh):
    return 3.14159 * ban_kinh ** 2


r = float(input())
print(f"{dien_tich_hinh_tron(r):.2f}")
```

Từ khóa `def` mở đầu định nghĩa; thân hàm phải thụt vào. `return` trả kết quả về nơi gọi và kết thúc hàm ngay; hàm không có `return` trả về `None`. **Hàm phải được định nghĩa trước khi gọi.**

2. Tham số, biến cục bộ, hàm Boolean

Tên trong dòng `def` là tham số; giá trị truyền vào khi gọi là đối số. Biến tạo bên trong hàm là **biến cục bộ**, tách biệt với biến cùng tên bên ngoài. Hàm trả `True`/`False` gọi là **hàm Boolean**; đặt tên để câu điều kiện đọc lên như một câu tiếng Việt. Có thể dùng kết quả hàm này làm đối số cho hàm khác (**composition**).

```
def la_nam_nhuan(y):
    return y % 400 == 0 or (y % 4 == 0 and y % 100 != 0)


if la_nam_nhuan(int(input())):
    print("NAM_NHUAN")
```

3. Chuỗi ký tự

Chuỗi là dãy ký tự, đánh chỉ số từ 0; chỉ số âm đếm từ cuối (`s[-1]` là ký tự cuối). Chuỗi **không thay đổi được**. Cú pháp cắt lát `s[bat_dau:ket_thuc]` lấy đến trước vị trí kết thúc; bước nhảy `-1` cho chuỗi đảo ngược.

```
s = "Python"
print(s[0], s[-1], len(s))
print(s[0:3])     # Pyt
print(s[::-1])    # nohtyP
```

4. Các phương thức chuỗi thông dụng

Phương thức	Tác dụng	Ví dụ
`s.upper()` / `s.lower()`	Chữ hoa / chữ thường	`"abc".upper()` → `"ABC"`
`s.strip()`	Bỏ dấu cách hai đầu	`"  a  ".strip()` → `"a"`
`s.split()`	Tách thành danh sách từ	`"a b".split()` → `["a","b"]`
`s.replace(a, b)`	Thay chuỗi con	`"a-b".replace("-","")` → `"ab"`
`s.count(x)` / `s.find(x)`	Đếm / vị trí xuất hiện	`"abab".count("a")` → `2`
`s.isalpha()` / `s.isdigit()`	Kiểm tra loại ký tự	`"123".isdigit()` → `True`
`" ".join(ds)`	Nối danh sách thành chuỗi	`" ".join(["a","b"])` → `"a b"`

`find` trả về `-1` khi không tìm thấy. Toán tử `in` cho biết một chuỗi con có xuất hiện hay không. Duyệt từng ký tự bằng `for ky_tu in s`.

Lưu ý và lỗi thường gặp

- Quên `return`, khiến hàm luôn trả về `None`.
- Gọi hàm nhưng không dùng kết quả (viết `tinh(x)` mà không gán hay in).
- Truy cập chỉ số vượt phạm vi (`IndexError`); nhớ chỉ số cuối là `len(s) - 1`.
- Dùng dấu cộng để nối chuỗi với số mà chưa `str()`.
- Quên rằng `split()` không tham số đã tự bỏ qua khoảng trắng thừa.

Ghi nhớ trước khi nộp bài

Định nghĩa hàm đặt trước phần chương trình chính; đặt tên hàm/biến gợi nghĩa; thử lại với chuỗi rỗng hoặc một ký tự; kiểm tra định dạng từng dòng kết quả.

Thực hành trên lớp

Bài 2.1: Hàm tính chu vi và diện tích hình chữ nhật

Chủ đề: Hàm, giá trị trả về

Viết hai hàm chu_vi(a, b) và dien_tich(a, b) trả về chu vi và diện tích của hình chữ nhật có hai cạnh a và b. Chương trình chính nhập a, b rồi in kết quả của hai hàm, mỗi giá trị trên một dòng, với 2 chữ số thập phân.

Input: Hai số thực a và b, cách nhau bởi dấu cách.

Output: Dòng 1: chu vi. Dòng 2: diện tích. Cả hai lấy 2 chữ số thập phân.

Code mẫu:

```
def chu_vi(a, b):
    return 2 * (a + b)


def dien_tich(a, b):
    return a * b


a, b = map(float, input().split())
print(f"{chu_vi(a, b):.2f}")
print(f"{dien_tich(a, b):.2f}")
```

Test case:

#	Input	Output
1	3 4	14.00 12.00
2	1.5 2.5	8.00 3.75
3	10 10	40.00 100.00
4	0.1 0.2	0.60 0.02
5	12.75 3	31.50 38.25

Hướng dẫn: Hai hàm nhận hai cạnh và trả về công thức tương ứng; chương trình chính gọi và in kết quả.

Bài 2.2: Hàm kiểm tra số nguyên tố

Chủ đề: Hàm Boolean

Viết hàm Boolean la_nguyen_to(n) trả về True nếu n là số nguyên tố. Chương trình chính nhập hai số nguyên a và b (a ≤ b), in tất cả các số nguyên tố thuộc đoạn [a, b] trên một dòng, cách nhau bởi dấu cách. Nếu không có số nào, in KHONG_CO.

Input: Hai số nguyên a và b, cách nhau bởi dấu cách.

Output: Danh sách số nguyên tố trên một dòng, hoặc KHONG_CO.

Code mẫu:

```
def la_nguyen_to(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True


a, b = map(int, input().split())
ket_qua = []
for x in range(a, b + 1):
    if la_nguyen_to(x):
        ket_qua.append(str(x))
if len(ket_qua) == 0:
    print("KHONG_CO")
else:
    print(" ".join(ket_qua))
```

Test case:

#	Input	Output
1	1 20	2 3 5 7 11 13 17 19
2	20 22	KHONG_CO
3	-5 3	2 3
4	90 100	97
5	97 97	97

Hướng dẫn: Đóng gói việc kiểm tra số nguyên tố thành một hàm trả True/False, tối ưu bằng điều kiện i*i <= n.

Bài 2.3: Rút gọn phân số

Chủ đề: Hàm, thuật toán Euclid

Nhập tử số a và mẫu số b (b khác 0) của một phân số. Rút gọn phân số về dạng tối giản và in theo mẫu tu/mau. Dấu âm nếu có luôn đặt ở tử số; nếu phân số bằng 0 thì in 0/1.

Input: Hai số nguyên a và b, cách nhau bởi dấu cách.

Output: Phân số tối giản dạng tu/mau.

Code mẫu:

```
def ucln(x, y):
    x = abs(x)
    y = abs(y)
    while y != 0:
```

```
        x, y = y, x % y
    return x


a, b = map(int, input().split())
if a == 0:
    print("0/1")
else:
    if b < 0:
        a = -a
        b = -b
    g = ucln(a, b)
    print(f"{a // g}/{b // g}")
```

Test case:

#	Input	Output
1	6 8	3/4
2	3 -9	-1/3
3	10 5	2/1
4	0 7	0/1
5	-4 6	-2/3

Hướng dẫn: Rút gọn bằng UCLN; đưa dấu âm về tử số và xử lý riêng phân số bằng 0.

Bài 2.4: Máy tính bỏ túi

Chủ đề: Chuỗi, rẽ nhánh

Nhập một biểu thức dạng a op b trên một dòng, trong đó a và b là số nguyên, op là một trong bốn phép +, -, *, /. In kết quả. Với phép chia, in kết quả 2 chữ số thập phân; nếu chia cho 0 thì in LOI_CHIA_0.

Input: Một dòng gồm ba phần cách nhau bởi dấu cách: a, op, b.

Output: Kết quả phép tính, hoặc LOI_CHIA_0.

Code mẫu:

```
a, op, b = input().split()
a = int(a)
b = int(b)
if op == "+":
    print(a + b)
elif op == "-":
    print(a - b)
elif op == "*":
    print(a * b)
else:
    if b == 0:
        print("LOI_CHIA_0")
    else:
        print(f"{a / b:.2f}")
```

Test case:

#	Input	Output
1	7 + 3	10
2	10 - 15	-5
3	6 * 7	42
4	7 / 2	3.50
5	5 / 0	LOI_CHIA_0

Hướng dẫn: Tách 'a op b', chọn phép tính theo op; xử lý chia cho 0 riêng.

Bài 2.5: Chuẩn hóa họ tên

Chủ đề: Chuỗi, split và join

Nhập một họ tên có thể chứa nhiều dấu cách thừa ở đầu, ở cuối và giữa các từ. In lại họ tên đã chuẩn hóa: mỗi từ cách nhau đúng một dấu cách, chữ cái đầu mỗi từ viết hoa, các chữ còn lại viết thường.

Input: Một dòng văn bản.

Output: Họ tên đã chuẩn hóa.

Code mẫu:

```
ho_ten = input()
tu = ho_ten.split()
ket_qua = []
for t in tu:
    ket_qua.append(t[0].upper() + t[1:].lower())
print(" ".join(ket_qua))
```

Test case:

#	Input	Output
1	`  nguyen   VAN  an`	`Nguyen Van An`
2	`TRAN THI BINH`	`Tran Thi Binh`
3	`le   hoang`	`Le Hoang`
4	`hoang anh TUAN`	`Hoang Anh Tuan`
5	`mai`	`Mai`

Hướng dẫn: split() bỏ khoảng trắng thừa, viết hoa chữ cái đầu mỗi từ rồi nối lại bằng join.

Bài 2.6: Thống kê từ trong câu

Chủ đề: Chuỗi, list

Nhập một câu. In số từ trong câu, từ dài nhất và độ dài của nó. Nếu có nhiều từ cùng độ dài lớn nhất, lấy từ xuất hiện đầu tiên. Các từ được phân tách bởi dấu cách.

Input: Một dòng văn bản có ít nhất một từ.

Output: Dòng 1: số từ. Dòng 2: từ dài nhất và độ dài của nó, cách nhau bởi dấu cách.

Code mẫu:

```
cau = input()
tu = cau.split()
dai_nhat = tu[0]
for t in tu:
    if len(t) > len(dai_nhat):
        dai_nhat = t
print(len(tu))
print(dai_nhat, len(dai_nhat))
```

Test case:

#	Input	Output
1	`Tin hoc dai cuong`	`4` `cuong 5`
2	`Python la ngon ngu lap trinh`	`6` `Python 6`
3	`  mot   hai  ba`	`3`

		mot 3
4	abc	1 abc 3
5	hoc lap trinh that thu vi	6 trinh 5

Hướng dẫn: Tách câu thành danh sách từ, tìm từ dài nhất bằng cách so sánh len.

Bài tập tự luyện (không bắt buộc)

Bài 2.7: Nút gốc chữ số

Chủ đề: Hàm, while lồng nhau

Nhập số nguyên dương n. Viết một hàm có chức năng cộng dồn các chữ số của n để được một số mới; lặp lại cho tới khi còn một chữ số duy nhất, rồi in ra. Ví dụ 9875 cho 9+8+7+5=29, rồi 2+9=11, rồi 1+1=2.

Input: Một số nguyên dương n.

Output: Chữ số cuối cùng thu được.

Code mẫu:

```
def tong_chu_so(x):
    tong = 0
    while x > 0:
        tong += x % 10
        x //= 10
    return tong


n = int(input())
while n >= 10:
    n = tong_chu_so(n)
print(n)
```

Test case:

#	Input	Output
1	5	5
2	9875	2
3	99999	9
4	10	1
5	123456789	9

Hướng dẫn: Một hàm tính tổng chữ số; lặp gọi hàm đó tới khi còn một chữ số duy nhất.

Bài 2.8: Đếm số chẵn bằng hàm

Chủ đề: Hàm, hàm Boolean

Viết hàm la_chan(n) trả về True nếu n là số chẵn. Chương trình chính nhập số nguyên dương m và m số nguyên, dùng hàm đó để đếm số lượng số chẵn trong dãy.

Input: Dòng 1: m. Dòng 2: m số nguyên.

Output: Số lượng số chẵn.

Code mẫu:

```
def la_chan(n):
```

```
    return n % 2 == 0


m = int(input())
so = list(map(int, input().split()))
dem = 0
for x in so:
    if la_chan(x):
        dem += 1
print(dem)
```

Test case:

#	Input	Output
1	5 1 2 3 4 5	2
2	3 2 4 6	3
3	3 1 3 5	0
4	1 0	1
5	4 -2 -1 0 1	2

Hướng dẫn: Đóng gói việc kiểm tra chẵn thành hàm trả True/False, rồi đếm bằng vòng lặp.

Bài 2.9: Hàm tính lũy thừa

Chủ đề: Hàm, vòng lặp

Viết hàm luy_thua(a, n) trả về a lũy thừa n bằng vòng lặp, không dùng toán tử . Chương trình chính nhập a và n rồi in kết quả. Quy ước a mũ 0 bằng 1.

Input: Hai số nguyên a và n (n ≥ 0), cách nhau bởi dấu cách.

Output: Giá trị a lũy thừa n.

Code mẫu:

```
def luy_thua(a, n):
    kq = 1
    for i in range(n):
        kq *= a
    return kq


a, n = map(int, input().split())
print(luy_thua(a, n))
```

Test case:

#	Input	Output
1	2 10	1024
2	3 0	1
3	5 3	125
4	10 2	100
5	-2 3	-8

Hướng dẫn: Nhân dồn a đúng n lần trong hàm; a mũ 0 = 1 nhờ vòng lặp chạy 0 lần.

Bài 2.10: Đếm số câu

Chủ đề: Chuỗi, đếm ký tự

Nhập một đoạn văn bản. Đếm số câu bằng cách đếm số ký tự kết thúc câu, gồm dấu chấm, dấu chấm than và dấu chấm hỏi.

Input: Một dòng văn bản.

Output: Số câu.

Code mẫu:

```
s = input()
dem = 0
for c in s:
    if c in ".!?":
        dem += 1
print(dem)
```

Test case:

#	Input	Output
1	Hello. How are you?	2
2	Toi di hoc. Ban di choi.	2
3	Khong co dau cau	0
4	A! B! C!	3
5	That vay sao?	1

Hướng dẫn: Đếm các ký tự kết thúc câu gồm dấu chấm, chấm than và chấm hỏi.

Bài 2.11: Phân loại chuỗi

Chủ đề: Chuỗi, rẽ nhánh

Nhập một chuỗi không rỗng. In SO nếu chuỗi chỉ gồm chữ số; CHU nếu chỉ gồm chữ cái; HON_HOP nếu chỉ gồm chữ cái và chữ số nhưng có cả hai loại; KHAC nếu có bất kỳ ký tự nào khác (kể cả dấu cách).

Input: Một dòng không rỗng.

Output: SO, CHU, HON_HOP hoặc KHAC.

Code mẫu:

```
s = input()
if s.isdigit():
    print("SO")
elif s.isalpha():
    print("CHU")
elif s.isalnum():
    print("HON_HOP")
else:
    print("KHAC")
```

Test case:

#	Input	Output
1	12345	SO
2	Python	CHU
3	abc123	HON_HOP
4	a b c	KHAC
5	12.5	KHAC

Hướng dẫn: Phân loại bằng isdigit/isalpha/isalnum theo thứ tự để tách SO, CHU, HON_HOP, KHAC.

Bài 2.12: Đếm số lần xuất hiện chuỗi con

Chủ đề: Chuỗi

Nhập chuỗi s ở dòng thứ nhất và chuỗi t ở dòng thứ hai. Đếm số lần chuỗi t xuất hiện trong s (các lần xuất hiện không chồng lấp), có phân biệt chữ hoa chữ thường.

Input: Dòng 1: chuỗi s. Dòng 2: chuỗi t.

Output: Số lần t xuất hiện trong s.

Code mẫu:

```
s = input()
t = input()
print(s.count(t))
```

Test case:

#	Input	Output
1	ababab ab	3
2	aaaa aa	2
3	hello l	2
4	abc x	0
5	Tin hoc tin tin	1

Hướng dẫn: Dùng s.count(t) đếm số lần chuỗi con xuất hiện không chồng lấp.

BÀI THỰC HÀNH SỐ 3: CHUỖI (TIẾP), TUPLE, LIST, SET, DICTIONARY

Tóm tắt kiến thức

Mục tiêu: Sau buổi thực hành, sinh viên dùng được list và tuple để lưu trữ, duyệt và biến đổi dữ liệu; làm quen với set và dictionary (nội dung nâng cao).

1. List

List là dãy giá trị có thứ tự và **thay đổi được**, đánh chỉ số từ 0.

```
diem = [8.5, 6.0, 9.25, 4.75]
print(diem[0], diem[-1], len(diem))
diem[1] = 7.0
```

Thao tác	Tác dụng
`ds.append(x) / ds.insert(i, x)`	Thêm vào cuối / chèn vào vị trí i
`ds.remove(x) / ds.pop()`	Xóa giá trị x / lấy và xóa phần tử cuối
`ds.sort() / ds.reverse()`	Sắp xếp tăng dần / đảo thứ tự (sửa trực tiếp)
`len(ds), sum(ds), max(ds), min(ds)`	Số phần tử, tổng, lớn nhất, nhỏ nhất
`x in ds`	Kiểm tra thuộc danh sách

Lưu ý `ds.sort()` **không trả về giá trị**; viết `ds = ds.sort()` sẽ làm mất dữ liệu.

2. Tuple

Tuple giống list nhưng **không thay đổi được**; thích hợp gom một nhóm thông tin thuộc cùng một đối tượng.

```
sinh_vien = ("An", 8.5)
ten, diem = sinh_vien          # giải nén tuple

lop = [("An", 8.5), ("Binh", 6.0)]
for ten, diem in lop:
    print(ten, diem)
```

Kỹ thuật gán nhiều biến như trên cũng chính là cơ chế của câu lệnh hoán đổi `a, b = b, a`.

3. Chuỗi và list làm việc cùng nhau

Hai phương thức `split` và `join` là cầu nối giữa chuỗi và list, xuất hiện trong hầu hết bài xử lý văn bản. Khi in một danh sách số, chuyển từng phần tử sang chuỗi trước: `" ".join(map(str, ds))`.

4. Set (nâng cao)

Set là tập hợp **không thứ tự, không trùng**. Các phép: `&` (giao), `|` (hợp), `-` (hiệu), `^` (đối xứng), `<=` (tập con), `a.isdisjoint(b)` (rời nhau). Vì không có thứ tự, luôn `sorted()` trước khi in để kết quả ổn định.

```
a = set(map(int, input().split()))
b = set(map(int, input().split()))
print(sorted(a & b))     # giao, tăng dần
print(len(set(ds)))      # số giá trị khác nhau
```

5. Dictionary (nâng cao)

Dictionary ánh xạ **khóa → giá trị**. Từ Python 3.7, dict giữ thứ tự thêm phần tử.

```
d = {}
for t in cau.split():
    d[t] = d.get(t, 0) + 1   # đếm tần suất
for k in d:
    print(k, d[k])
print(d.get(key, "KHONG_CO"))  # tra cứu có mặc định
```

Lưu ý và lỗi thường gặp

- `ds.sort()` trả về `None`; đừng gán lại. Đừng sửa list trong khi đang duyệt nó.
- Truy cập chỉ số vượt phạm vi (`IndexError`).
- Set không có thứ tự → nhớ `sorted()`; dictionary in theo thứ tự thêm.
- Truy cập khóa không tồn tại bằng `d[k]` gây `KeyError`; dùng `d.get(k, mặc_định)`.

Ghi nhớ trước khi nộp bài

Thử lại bài với dữ liệu chỉ một phần tử và với dữ liệu có phần tử trùng nhau; kiểm tra kỹ định dạng từng dòng kết quả.

Thực hành trên lớp

Bài 3.1: Đổi sang hệ nhị phân (Chuỗi)

Chủ đề: Chuỗi

Nhập số nguyên không âm n. In biểu diễn nhị phân của n mà không dùng hàm bin. Quy ước số 0 in ra 0.

Input: Một số nguyên không âm n.

Output: Chuỗi nhị phân của n.

Code mẫu:

```
n = int(input())
if n == 0:
    print("0")
else:
    ket_qua = ""
    while n > 0:
        ket_qua = str(n % 2) + ket_qua
        n //= 2
    print(ket_qua)
```

Test case:

#	Input	Output
1	0	0
2	1	1
3	5	101
4	10	1010
5	255	11111111

Hướng dẫn: Lặp lấy dư 2 rồi ghép vào đầu chuỗi kết quả, đồng thời chia đôi n.

Bài 3.2: Bảng điểm lớp học (Tuple)

Chủ đề: Tuple

Nhập số sinh viên n. Mỗi dòng tiếp theo gồm họ tên (không chứa dấu cách) và điểm số thực, cách nhau bởi dấu cách. Lưu mỗi sinh viên thành một tuple trong danh sách. In điểm trung bình của lớp (2 chữ số thập phân), tên sinh viên có điểm cao nhất, và số sinh viên đạt (điểm ≥ 5). Nếu nhiều sinh viên cùng điểm cao nhất, lấy người xuất hiện đầu tiên.

Input: Dòng 1: số nguyên dương n. n dòng tiếp theo: tên và điểm.

Output: Ba dòng: điểm trung bình, tên sinh viên điểm cao nhất, số sinh viên đạt.

Code mẫu:

```
n = int(input())
lop = []
for i in range(n):
    ten, diem = input().split()
    lop.append((ten, float(diem)))

tong = 0
cao_nhat = lop[0]
dat = 0
for ten, diem in lop:
    tong += diem
    if diem > cao_nhat[1]:
        cao_nhat = (ten, diem)
    if diem >= 5:
        dat += 1
print(f"{tong / n:.2f}")
print(cao_nhat[0])
print(dat)
```

Test case:

#	Input	Output
1	3	6.33

	An 8.5 Binh 6 Cuong 4.5	An 2
2	1 Dung 10	10.00 Dung 1
3	4 An 5 Binh 5 Cuong 5 Dung 5	5.00 An 4
4	5 A 9.25 B 3 C 7.5 D 4.75 E 9.25	6.75 A 3
5	2 Mai 0 Nam 2.5	1.25 Nam 0

Hướng dẫn: Lưu mỗi sinh viên thành tuple (tên, điểm); duyệt để tính trung bình, tìm cao nhất và đếm đạt.

Bài 3.3: Kiểm tra mật khẩu mạnh (List)

Chủ đề: List

Nhập một chuỗi mật khẩu (không chứa dấu cách). Mật khẩu được coi là mạnh nếu thỏa cả bốn điều kiện: dài ít nhất 8 ký tự, có ít nhất một chữ hoa, một chữ thường và một chữ số. In MANH hoặc YEU.

Input: Một chuỗi không chứa dấu cách.

Output: MANH hoặc YEU.

Code mẫu:

```
def manh(mk):
    if len(mk) < 8:
        return False
    co_hoa = False
    co_thuong = False
    co_so = False
    for c in mk:
        if c.isupper():
            co_hoa = True
        elif c.islower():
            co_thuong = True
        elif c.isdigit():
            co_so = True
    return co_hoa and co_thuong and co_so


mat_khau = input()
if manh(mat_khau):
    print("MANH")
else:
    print("YEU")
```

Test case:

#	Input	Output

1	Abcdef12	MANH
2	abc123	YEU
3	ABCDEFGH	YEU
4	Python2025	MANH
5	aB3xY9zzz	MANH

Hướng dẫn: Duyệt ký tự, dùng cờ cho chữ hoa/thường/số; kiểm tra đủ bốn điều kiện cùng độ dài.

Bài 3.4: Loại bỏ phần tử trùng lặp (Set)

Chủ đề: Set

Nhập số nguyên dương n và n số nguyên. In danh sách các giá trị khác nhau theo đúng thứ tự xuất hiện lần đầu, sau đó in số lượng giá trị khác nhau.

Input: Dòng 1: số nguyên dương n. Dòng 2: n số nguyên cách nhau bởi dấu cách.

Output: Dòng 1: các giá trị khác nhau. Dòng 2: số lượng giá trị khác nhau.

Code mẫu:

```
n = int(input())
danh_sach = list(map(int, input().split()))
ket_qua = []
for x in danh_sach:
    if x not in ket_qua:
        ket_qua.append(x)
print(" ".join(map(str, ket_qua)))
print(len(ket_qua))
```

Test case:

#	Input	Output
1	6 1 2 2 3 1 4	1 2 3 4 4
2	5 7 7 7 7 7	7 1
3	4 1 2 3 4	1 2 3 4 4
4	1 0	0 1
5	7 -1 3 -1 0 3 5 0	-1 3 0 5 4

Hướng dẫn: Duyệt và chỉ thêm phần tử chưa có trong danh sách kết quả để giữ thứ tự xuất hiện đầu.

Bài 3.5: Sổ điểm có cập nhật (Dictionary)

Chủ đề: Dictionary

Nhập số nguyên dương n, sau đó n dòng, mỗi dòng gồm tên (không chứa dấu cách) và điểm số thực; nếu một tên xuất hiện nhiều lần thì lấy điểm nhập sau cùng. Dòng cuối là một tên cần tra cứu. In điểm của tên đó với 2 chữ số thập phân, hoặc KHONG_CO nếu không có.

Input: Dòng 1: n. n dòng: tên và điểm. Dòng cuối: tên cần tra.

Output: Điểm với 2 chữ số thập phân, hoặc KHONG_CO.

Code mẫu:

```
n = int(input())
diem = {}
```

```
for i in range(n):
    ten, d = input().split()
    diem[ten] = float(d)
can_tra = input().strip()
if can_tra in diem:
    print(f"{diem[can_tra]:.2f}")
else:
    print("KHONG_CO")
```

Test case:

#	Input	Output
1	3 An 8 Binh 6 An 9 An	9.00
2	2 An 7 Binh 5 Cuong	KHONG_CO
3	1 Mai 10 Mai	10.00
4	3 A 5 B 6 C 7 B	6.00
5	2 X 8.5 X 9.5 X	9.50

Hướng dẫn: Ghi đè điểm khi tên trùng; dùng get để tra cứu.

Bài tập tự luyện (không bắt buộc)

Bài 3.6: Kiểm tra chuỗi đối xứng

Chủ đề: Chuỗi, cắt và đảo

Nhập một chuỗi. Kiểm tra chuỗi có đối xứng hay không sau khi bỏ hết dấu cách và không phân biệt chữ hoa chữ thường. Viết hàm doi_xung(s) trả về giá trị Boolean.

Input: Một dòng văn bản.

Output: DOI_XUNG hoặc KHONG_DOI_XUNG.

Code mẫu:

```
def doi_xung(s):
    s = s.replace(" ", "").lower()
    return s == s[::-1]


van_ban = input()
if doi_xung(van_ban):
    print("DOI_XUNG")
else:
    print("KHONG_DOI_XUNG")
```

Test case:

#	Input	Output
1	aba	DOI_XUNG
2	Python	KHONG_DOI_XUNG
3	Nam co Nam con	KHONG_DOI_XUNG
4	abba	DOI_XUNG
5	Toi di hoc	KHONG_DOI_XUNG

Hướng dẫn: Bỏ dấu cách và hạ chữ thường, so sánh chuỗi với bản đảo ngược s[::-1].

Bài 3.7: Đếm nguyên âm và phụ âm (chuỗi)

Chủ đề: Chuỗi

Nhập một chuỗi bất kỳ. Đếm số nguyên âm (a, e, i, o, u, không phân biệt chữ hoa chữ thường), số chữ cái khác và số ký tự không phải chữ cái.

Input: Một dòng văn bản.

Output: Ba số nguyên: số nguyên âm, số phụ âm, số ký tự khác; cách nhau bởi dấu cách.

Code mẫu:

```
s = input()
nguyen_am = 0
phu_am = 0
khac = 0
for ky_tu in s:
    if ky_tu.isalpha():
        if ky_tu.lower() in "aeiou":
            nguyen_am += 1
        else:
            phu_am += 1
    else:
        khac += 1
print(nguyen_am, phu_am, khac)
```

Test case:

#	Input	Output
1	Hello World	3 7 1
2	PYTHON	1 5 0
3	12345	0 0 5
4	Tin hoc dai cuong 2025	6 8 8
5	aeiou AEIOU	10 0 1

Hướng dẫn: Duyệt từng ký tự, dùng isalpha và kiểm tra ký tự thuộc 'aeiou' để phân loại nguyên âm/phụ âm.

Bài 3.8: Thống kê trong một câu

Chủ đề: Chuỗi, list

Nhập một câu. In số từ trong câu, từ dài nhất (nếu nhiều từ cùng độ dài lớn nhất thì lấy từ đầu tiên), và câu sau khi viết hoa chữ cái đầu mỗi từ, các chữ còn lại viết thường. Ba kết quả trên ba dòng.

Input: Một dòng văn bản có ít nhất một từ.

Output: Dòng 1: số từ. Dòng 2: từ dài nhất. Dòng 3: câu đã chuẩn hóa hoa đầu từ.

Code mẫu:

```
cau = input()
tu = cau.split()
dai_nhat = tu[0]
for t in tu:
    if len(t) > len(dai_nhat):
        dai_nhat = t
chuan = []
for t in tu:
    chuan.append(t[0].upper() + t[1:].lower())
print(len(tu))
print(dai_nhat)
print(" ".join(chuan))
```

Test case:

#	Input	Output
1	tin hoc dai cuong	4 cuong Tin Hoc Dai Cuong
2	PYTHON la thu vi	4 PYTHON Python La Thu Vi
3	mot hai ba	3 mot Mot Hai Ba
4	hello	1 hello Hello
5	lap trinh huong doi tuong	5 trinh Lap Trinh Huong Doi Tuong

Hướng dẫn: Đếm số từ, tìm từ dài nhất và chuẩn hóa viết hoa chữ đầu mỗi từ.

Bài 3.9: Kiểm tra tính đơn điệu

Chủ đề: List, so sánh liền kề

Nhập số nguyên n và n số nguyên. In TANG nếu dãy tăng dần nghiêm ngặt, GIAM nếu giảm dần nghiêm ngặt, và KHONG nếu không thuộc hai loại trên. Dãy một phần tử coi là TANG.

Input: Dòng 1: n. Dòng 2: n số nguyên.

Output: TANG, GIAM hoặc KHONG.

Code mẫu:

```
n = int(input())
so = list(map(int, input().split()))
tang = True
giam = True
for i in range(1, n):
    if so[i] <= so[i - 1]:
        tang = False
    if so[i] >= so[i - 1]:
        giam = False
if tang:
    print("TANG")
elif giam:
    print("GIAM")
else:
```

```
    print("KHONG")
```

Test case:

#	Input	Output
1	5 1 2 3 4 5	TANG
2	4 9 7 3 1	GIAM
3	5 1 3 2 4 5	KHONG
4	1 7	TANG
5	3 5 5 5	KHONG

Hướng dẫn: Dùng hai cờ tăng/giảm, duyệt so sánh mỗi phần tử với phần tử liền trước.

Bài 3.10: Trộn hai dãy đã sắp xếp

Chủ đề: List, hai con trỏ

Cho hai dãy số nguyên đã sắp xếp tăng dần. Trộn chúng thành một dãy tăng dần duy nhất mà không dùng hàm sort. Dòng 1 và dòng 2 lần lượt là hai dãy; nếu một dãy rỗng thì dòng tương ứng để trống.

Input: Hai dòng, mỗi dòng là một dãy số nguyên tăng dần cách nhau bởi dấu cách.

Output: Dãy sau khi trộn, các số cách nhau bởi dấu cách.

Code mẫu:

```
a = input().split()
b = input().split()
a = [int(x) for x in a]
b = [int(x) for x in b]
i = 0
j = 0
ket_qua = []
while i < len(a) and j < len(b):
    if a[i] <= b[j]:
        ket_qua.append(a[i])
        i += 1
    else:
        ket_qua.append(b[j])
        j += 1
while i < len(a):
    ket_qua.append(a[i])
    i += 1
while j < len(b):
    ket_qua.append(b[j])
    j += 1
print(" ".join(map(str, ket_qua)))
```

Test case:

#	Input	Output
1	1 3 5 2 4 6	1 2 3 4 5 6
2	1 2 3 4 5 6	1 2 3 4 5 6
3	 1 2 3	1 2 3

4	5 1 9	1 5 9
5	1 4 4 7 2 4 8	1 2 4 4 4 7 8

Hướng dẫn: Dùng hai chỉ số duyệt song song hai dãy đã sắp, luôn lấy phần tử nhỏ hơn trước.

Bài 3.11: Tính trung bình sau khi loại đỉnh

Chủ đề: List, hàm có sẵn

Nhập số nguyên n (n ≥ 3) và n số nguyên. Loại bỏ đúng một giá trị lớn nhất và một giá trị nhỏ nhất (mỗi loại một lần), rồi in trung bình cộng của các số còn lại với 2 chữ số thập phân.

Input: Dòng 1: n. Dòng 2: n số nguyên cách nhau bởi dấu cách.

Output: Trung bình cộng sau khi loại đỉnh, 2 chữ số thập phân.

Code mẫu:

```
n = int(input())
so = list(map(int, input().split()))
tong = sum(so) - max(so) - min(so)
print(f"{tong / (n - 2):.2f}")
```

Test case:

#	Input	Output
1	3 1 2 3	2.00
2	5 10 2 8 4 6	6.00
3	4 5 5 5 5	5.00
4	6 1 1 1 9 9 9	5.00
5	3 -5 0 5	0.00

Hướng dẫn: Tổng trừ đi max và min rồi chia cho (n−2).

Bài 3.12: Xử lý danh sách số (LIST)

Chủ đề: List

Nhập số nguyên dương n và n số nguyên trên dòng tiếp theo. Tạo danh sách, sau đó in: tổng của danh sách, giá trị lớn nhất, số lượng phần tử chẵn và danh sách sau khi sắp xếp tăng dần. Được phép dùng các hàm có sẵn của Python.

Input: Dòng 1: số nguyên dương n. Dòng 2: n số nguyên cách nhau bởi dấu cách.

Output: Bốn dòng: tổng, giá trị lớn nhất, số phần tử chẵn, danh sách đã sắp xếp.

Code mẫu:

```
n = int(input())
danh_sach = list(map(int, input().split()))
dem_chan = 0
for x in danh_sach:
    if x % 2 == 0:
        dem_chan += 1
danh_sach.sort()
print(sum(danh_sach))
```

```
print(max(danh_sach))
print(dem_chan)
print(" ".join(map(str, danh_sach)))
```

Test case:

#	Input	Output
1	5 5 3 8 1 9	26 9 1 1 3 5 8 9
2	1 42	42 42 1 42
3	6 -4 7 0 -1 2 2	6 7 4 -4 -1 0 2 2 7
4	4 10 20 30 40	100 40 4 10 20 30 40
5	3 -5 -7 -9	-21 -5 0 -9 -7 -5

Hướng dẫn: Dùng sum/max, đếm số chẵn bằng vòng lặp, và sort() để sắp xếp danh sách.

Bài 3.13: Tìm kiếm vị trí xuất hiện

Chủ đề: List, chỉ số

Nhập số nguyên dương n, n số nguyên và một giá trị cần tìm k. In tất cả các vị trí (đánh số từ 0) mà k xuất hiện trong danh sách, cách nhau bởi dấu cách. Nếu k không có mặt, in KHONG_TIM_THAY. Không dùng phương thức index.

Input: Dòng 1: n. Dòng 2: n số nguyên. Dòng 3: giá trị k.

Output: Danh sách vị trí, hoặc KHONG_TIM_THAY.

Code mẫu:

```
n = int(input())
danh_sach = list(map(int, input().split()))
k = int(input())
vi_tri = []
for i in range(n):
    if danh_sach[i] == k:
        vi_tri.append(str(i))
if len(vi_tri) == 0:
    print("KHONG_TIM_THAY")
else:
    print(" ".join(vi_tri))
```

Test case:

#	Input	Output
1	5 1 2 3 2 1 2	1 3

2	5 1 2 3 4 5 9	KHONG_TIM_THAY
3	4 7 7 7 7 7	0 1 2 3
4	1 0 0	0
5	6 -3 5 -3 8 -3 5 -3	0 2 4

Hướng dẫn: Duyệt theo chỉ số, ghi lại mọi vị trí có giá trị bằng giá trị cần tìm.

BÀI THỰC HÀNH SỐ 4: MODULE VÀ TẬP TIN

Tóm tắt kiến thức

Mục tiêu: Sau buổi thực hành, sinh viên sử dụng được module, đọc và ghi tệp văn bản, xử lý dữ liệu theo dòng và theo trường, và xử lý được tình huống tệp không tồn tại.

1. Module và thư viện

Module là mã viết sẵn để tái sử dụng. Thư viện chuẩn (`math`, `random`, `os`, `statistics`...) dùng ngay; gói bên thứ ba cài thêm bằng `pip`.

```
import math
from math import sqrt
import matplotlib.pyplot as plt   # đặt tên rút gọn
```

2. Tệp văn bản và cách mở

Tệp văn bản chứa ký tự đọc được, chia thành dòng (`.txt`, `.csv`). Khi có tiếng Việt, luôn ghi rõ `encoding="utf-8"`. Cách được khuyến nghị là dùng `with` để tệp tự đóng.

Chế độ	Ý nghĩa	Lưu ý
`"r"`	Đọc (mặc định)	Báo lỗi nếu tệp không tồn tại
`"w"`	Ghi mới	Xóa sạch nội dung cũ khi mở
`"a"`	Ghi nối vào cuối	Giữ nguyên nội dung cũ

```
with open("dulieu.txt", "r", encoding="utf-8") as f:
    for dong in f:
        dong = dong.rstrip("\n")   # bỏ ký tự xuống dòng
        print(dong)
```

Có thể đọc cả tệp bằng `f.read()` hoặc thành danh sách dòng bằng `f.readlines()`.

3. Ghi dữ liệu ra tệp

`f.write` ghi đúng chuỗi truyền vào và **không** tự thêm ký tự xuống dòng; phải tự viết `\n`. Dữ liệu số phải `str()` trước khi ghi.

```
with open("ketqua.txt", "w", encoding="utf-8") as f:
    for i in range(len(diem)):
        f.write(f"Sinh vien {i + 1}: {diem[i]:.2f}\n")
```

4. Tách dữ liệu theo trường (CSV)

Mỗi dòng là một bản ghi, các trường cách nhau bởi dấu phẩy. Dùng `split(",")` để tách, bỏ dòng tiêu đề, ép kiểu cho trường số.

```
with open("diem.csv", "r", encoding="utf-8") as f:
    dong = f.readlines()
for i in range(1, len(dong)):          # bỏ dòng tiêu đề
    truong = dong[i].strip().split(",")
    tb = (float(truong[1]) + float(truong[2])) / 2
```

5. Xử lý lỗi

Chương trình đọc tệp phải tính đến khả năng tệp không tồn tại. Khối `try / except` bắt lỗi thay vì để chương trình dừng đột ngột.

```
try:
    with open("dulieu.txt", "r", encoding="utf-8") as f:
        noi_dung = f.read()
except FileNotFoundError:
    print("KHONG_TIM_THAY_TEP")
```

6. An toàn và bản quyền khi dùng dữ liệu số

- Sao lưu dữ liệu gốc trước khi chạy chương trình có thao tác ghi hoặc xóa.
- Không mở/chạy tệp từ nguồn không rõ ràng; kiểm tra dữ liệu đọc từ tệp trước khi dùng.
- Không đưa thông tin cá nhân của người khác vào bài; ghi rõ nguồn và giấy phép của tập dữ liệu lấy trên mạng.

Lưu ý và lỗi thường gặp

- Mở chế độ `"w"` khi chỉ muốn đọc → mất toàn bộ dữ liệu.
- Quên `rstrip("\n")` nên chuỗi còn ký tự xuống dòng, so sánh luôn sai.
- Xử lý cả dòng tiêu đề CSV như dữ liệu; quên ép kiểu cho trường số.
- Đọc hết tệp một lần rồi đọc lại mà không mở lại (lần hai không còn dữ liệu).
- Sai tên tệp hoặc sai thư mục làm việc (`FileNotFoundError`).

Ghi nhớ trước khi nộp bài

Nộp kèm tệp dữ liệu mẫu; dùng đường dẫn tương đối và đúng tên tệp; chạy lại hai lần liên tiếp để chắc kết quả không bị nhân đôi; mở tệp kết quả lên xem trực tiếp.

Thực hành trên lớp

Bài 4.1: Thống kê cơ bản một tệp văn bản

Chủ đề: Đọc tệp, try/except

Đọc tệp dulieu.txt trong cùng thư mục với chương trình. In số dòng, số từ và số ký tự của tệp, mỗi giá trị trên một dòng. Không tính ký tự xuống dòng vào số ký tự. Nếu tệp không tồn tại, in KHONG_TIM_THAY_TEP.

Input: Tệp dulieu.txt (mã hóa UTF-8).

Output: Ba dòng: số dòng, số từ, số ký tự.

Tệp: dulieu.txt

```
Tin hoc dai cuong
Lap trinh Python
```

Code mẫu:

```
try:
    f = open("dulieu.txt", "r", encoding="utf-8")
except FileNotFoundError:
    print("KHONG_TIM_THAY_TEP")
else:
    so_dong = 0
    so_tu = 0
    so_ky_tu = 0
    for dong in f:
        dong = dong.rstrip("\n")
        so_dong += 1
        so_tu += len(dong.split())
        so_ky_tu += len(dong)
    f.close()
    print(so_dong)
    print(so_tu)
    print(so_ky_tu)
```

Test case:

#	Input	Output
1	[dulieu.txt] Tin hoc dai cuong Lap trinh Python	2 7 33
2	[dulieu.txt] Mot dong duy nhat	1 4 17
3	[dulieu.txt] a b c d	3 4 6
4	[dulieu.txt]	0 0 0
5	(khong co)	KHONG_TIM_THAY_TEP

Hướng dẫn: Mở tệp trong khối try và bắt FileNotFoundError; duyệt từng dòng để đếm dòng, từ, ký tự.

Bài 4.2: Ghi danh sách vào tệp

Chủ đề: Ghi tệp, with

Nhập số nguyên dương n, sau đó nhập n dòng văn bản. Ghi n dòng đó vào tệp ketqua.txt, mỗi dòng được đánh số theo mẫu i. noi_dung (bắt đầu từ 1). Sau khi ghi xong, in DA_GHI kèm số dòng đã ghi.

Input: Dòng 1: số nguyên dương n. n dòng tiếp theo: nội dung.

Output: Một dòng DA_GHI n trên màn hình và tệp ketqua.txt gồm n dòng đã đánh số.

Lưu ý: sau khi chương trình chạy xong, hệ thống sẽ tự động mở tệp `ketqua.txt` và in nội dung của tệp ra để đối chiếu. Bài làm không cần tự in nội dung tệp này.

Code mẫu:

```
n = int(input())
with open("ketqua.txt", "w", encoding="utf-8") as f:
    for i in range(1, n + 1):
        noi_dung = input()
        f.write(f"{i}. {noi_dung}\n")
```

```
print("DA_GHI", n)
```

Test case:

#	Input	Output
1	2 Hoc Python Hoc thuat toan	DA_GHI 2 --- ketqua.txt --- 1. Hoc Python 2. Hoc thuat toan
2	1 Mot dong	DA_GHI 1 --- ketqua.txt --- 1. Mot dong
3	3 a b c	DA_GHI 3 --- ketqua.txt --- 1. a 2. b 3. c

Hướng dẫn: Mở tệp chế độ 'w', ghi từng dòng đã đánh số kèm ký tự xuống dòng.

Bài 4.3: Tính toán trên tệp số

Chủ đề: Đọc tệp, list

Tệp so.txt chứa nhiều dòng, mỗi dòng một số nguyên. Đọc tệp và in tổng, giá trị lớn nhất, giá trị nhỏ nhất và trung bình cộng (2 chữ số thập phân). Bỏ qua các dòng trống. Nếu tệp không có số nào, in TEP_RONG.

Input: Tệp so.txt.

Output: Bốn dòng: tổng, lớn nhất, nhỏ nhất, trung bình cộng; hoặc TEP_RONG.

Tệp: so.txt

```
1
2
3
4
5
```

Code mẫu:

```
danh_sach = []
with open("so.txt", "r", encoding="utf-8") as f:
    for dong in f:
        dong = dong.strip()
        if dong != "":
            danh_sach.append(int(dong))

if len(danh_sach) == 0:
    print("TEP_RONG")
else:
    print(sum(danh_sach))
    print(max(danh_sach))
    print(min(danh_sach))
    print(f"{sum(danh_sach) / len(danh_sach):.2f}")
```

Test case:

#	Input	Output
1	[so.txt] 1 2	15 5 1

	3 4 5	3.00
2	[so.txt] 10 -4 7	13 10 -4 4.33
3	[so.txt] 42	42 42 42 42.00
4	[so.txt]	TEP_RONG
5	[so.txt] -1 -2 -3	-6 -1 -3 -2.00

Hướng dẫn: Đọc từng dòng, bỏ dòng trống, ép sang int; xử lý riêng trường hợp tệp không có số nào.

Bài 4.4: Tìm dòng chứa từ khóa

Chủ đề: Đọc tệp, chuỗi

Nhập một từ khóa từ bàn phím, sau đó đọc tệp vanban.txt. In số thứ tự và nội dung của mọi dòng có chứa từ khóa (không phân biệt chữ hoa chữ thường) theo mẫu i: noi_dung, với i bắt đầu từ 1. Cuối cùng in tổng số dòng tìm được theo mẫu TIM_THAY k.

Input: Dòng nhập: từ khóa. Tệp vanban.txt.

Output: Các dòng khớp và một dòng TIM_THAY k.

Tệp: vanban.txt

```
Hoc Python co ban
Thuat toan
PYTHON nang cao
```

Code mẫu:

```
tu_khoa = input().strip().lower()
dem = 0
with open("vanban.txt", "r", encoding="utf-8") as f:
    so_dong = 0
    for dong in f:
        so_dong += 1
        dong = dong.rstrip("\n")
        if tu_khoa in dong.lower():
            print(f"{so_dong}: {dong}")
            dem += 1
print("TIM_THAY", dem)
```

Test case:

#	Input	Output
1	[vanban.txt] Hoc Python co ban Thuat toan PYTHON nang cao	1: Hoc Python co ban 3: PYTHON nang cao TIM_THAY 2

	[ban phim] python	
2	[vanban.txt] Hoc Python Thuat toan [ban phim] java	TIM_THAY 0
3	[vanban.txt] abc xyz AAA [ban phim] a	1: abc 3: AAA TIM_THAY 2
4	[vanban.txt] Tin hoc dai cuong tin hoc [ban phim] tin hoc	1: Tin hoc dai cuong 2: tin hoc TIM_THAY 2

Hướng dẫn: Duyệt từng dòng, so khớp không phân biệt hoa thường bằng lower(), đếm số dòng khớp.

Bài 4.5: Đếm tần suất từ

Chủ đề: Đọc tệp, hai list song song

Đọc tệp vanban.txt, tách toàn bộ nội dung thành các từ (phân tách bởi dấu cách và ký tự xuống dòng), chuyển về chữ thường. Đếm số lần xuất hiện của mỗi từ và in ra theo thứ tự xuất hiện lần đầu, mỗi dòng theo mẫu tu so_lan. Gợi ý: dùng hai danh sách song song, một danh sách lưu từ và một danh sách lưu số đếm tương ứng.

Input: Tệp vanban.txt.

Output: Mỗi dòng gồm một từ và số lần xuất hiện.

Tệp: vanban.txt

```
hoc hoc nua hoc mai
```

Code mẫu:

```
tu = []
so_lan = []
with open("vanban.txt", "r", encoding="utf-8") as f:
    noi_dung = f.read()

for t in noi_dung.lower().split():
    if t in tu:
        vi_tri = tu.index(t)
        so_lan[vi_tri] += 1
    else:
        tu.append(t)
        so_lan.append(1)

for i in range(len(tu)):
    print(tu[i], so_lan[i])
```

Test case:

#	Input	Output
1	[vanban.txt] hoc hoc nua hoc mai	hoc 3 nua 1 mai 1

2	[vanban.txt] Python python PYTHON	python 3
3	[vanban.txt] a b c b c c	a 1 b 2 c 3
4	[vanban.txt] Hoc HOC hoc	hoc 3

Hướng dẫn: Đọc cả tệp rồi tách từ; đếm bằng một list từ và một list số đếm song song, không dùng dict.

Bài tập tự luyện (không bắt buộc)

Bài 4.6: Chuẩn hóa văn bản

Chủ đề: Đọc–ghi đồng thời

Đọc tệp thô goc.txt, bỏ các dòng trống, bỏ dấu cách thừa ở hai đầu mỗi dòng và thay nhiều dấu cách liên tiếp giữa các từ bằng đúng một dấu cách. Ghi kết quả vào tệp chuanhoa.txt và in số dòng đã ghi.

Input: Tệp goc.txt.

Output: Một số nguyên là số dòng đã ghi, và tệp chuanhoa.txt.

Lưu ý: sau khi chương trình chạy xong, hệ thống sẽ tự động mở tệp `chuanhoa.txt` và in nội dung của tệp ra để đối chiếu. Bài làm không cần tự in nội dung tệp này.

Tệp: goc.txt

```
  Tin   hoc

   dai cuong
```

Code mẫu:

```
dem = 0
with open("goc.txt", "r", encoding="utf-8") as f_in:
    with open("chuanhoa.txt", "w", encoding="utf-8") as f_out:
        for dong in f_in:
            tu = dong.split()
            if len(tu) > 0:
                f_out.write(" ".join(tu) + "\n")
                dem += 1
print(dem)
```

Test case:

#	Input	Output
1	[goc.txt] Tin hoc dai cuong	2 --- chuanhoa.txt --- Tin hoc dai cuong
2	[goc.txt] a b c	1 --- chuanhoa.txt --- a b c
3	[goc.txt]	0 --- chuanhoa.txt ---

Hướng dẫn: Mở đồng thời tệp nguồn và tệp đích; bỏ dòng trống và gộp khoảng trắng bằng split rồi join.

Bài 4.7: Gộp 2 tệp

Chủ đề: Đọc nhiều tệp, xử lý lỗi

Đọc hai tệp a.txt và b.txt, ghi toàn bộ nội dung của a.txt rồi đến b.txt vào tệp gop.txt. In tổng số dòng của tệp kết quả. Nếu một trong hai tệp không tồn tại, in THIEU_TEP và không tạo tệp kết quả.

Input: Hai tệp a.txt và b.txt.

Output: Tổng số dòng của tệp gộp, hoặc THIEU_TEP.

Lưu ý: sau khi chương trình chạy xong, hệ thống sẽ tự động mở tệp `gop.txt` và in nội dung của tệp ra để đối chiếu. Bài làm không cần tự in nội dung tệp này.

Danh sách các tệp:

a.txt

```
1
2
```

b.txt

```
3
```

Code mẫu:

```
try:
    with open("a.txt", "r", encoding="utf-8") as f:
        dong_a = f.readlines()
    with open("b.txt", "r", encoding="utf-8") as f:
        dong_b = f.readlines()
except FileNotFoundError:
    print("THIEU_TEP")
else:
    tat_ca = dong_a + dong_b
    with open("gop.txt", "w", encoding="utf-8") as f:
        for dong in tat_ca:
            if not dong.endswith("\n"):
                dong += "\n"
            f.write(dong)
    print(len(tat_ca))
```

Test case:

#	Input	Output
1	[a.txt] 1 2 [b.txt] 3	3 --- gop.txt --- 1 2 3
2	[a.txt] x [b.txt] y	2 --- gop.txt --- x y
3	[a.txt] chi co a	THIEU_TEP --- gop.txt: khong duoc tao ---

Hướng dẫn: Đọc readlines hai tệp rồi nối; bắt lỗi thiếu tệp; bảo đảm mỗi dòng kết thúc bằng ký tự xuống dòng.

Bài 4.8: Bảng điểm dạng CSV

Chủ đề: Đọc CSV, split(',')

Tệp diem.csv có dòng đầu là tiêu đề ho_ten,toan,ly,hoa và các dòng sau là dữ liệu, các trường cách nhau bởi dấu phẩy. Tính điểm trung bình ba môn của từng sinh viên và in mỗi sinh viên trên một dòng theo mẫu ho_ten diem_tb, với điểm trung bình lấy 2 chữ số thập phân. Cuối cùng in điểm trung bình của cả lớp theo mẫu LOP diem_tb.

Input: Tệp diem.csv.

Output: Mỗi sinh viên một dòng và một dòng tổng kết của lớp.

Tệp: diem.csv

```
ho_ten,toan,ly,hoa
An,8,7,9
Binh,5,6,4
```

Code mẫu:

```
with open("diem.csv", "r", encoding="utf-8") as f:
    dong = f.readlines()

tong_lop = 0
so_sv = 0
for i in range(1, len(dong)):
    noi_dung = dong[i].strip()
    if noi_dung == "":
        continue
    truong = noi_dung.split(",")
    ho_ten = truong[0]
    diem_tb = (float(truong[1]) + float(truong[2]) + float(truong[3])) / 3
    print(f"{ho_ten} {diem_tb:.2f}")
    tong_lop += diem_tb
    so_sv += 1

print(f"LOP {tong_lop / so_sv:.2f}")
```

Test case:

#	Input	Output
1	[diem.csv] ho_ten,toan,ly,hoa An,8,7,9 Binh,5,6,4	An 8.00 Binh 5.00 LOP 6.50
2	[diem.csv] ho_ten,toan,ly,hoa Cuong,10,10,10	Cuong 10.00 LOP 10.00
3	[diem.csv] ho_ten,toan,ly,hoa A,4.5,5.5,6 B,0,0,0 C,9.25,8,7.75	A 5.33 B 0.00 C 8.33 LOP 4.56

Hướng dẫn: Bỏ dòng tiêu đề, tách trường bằng dấu phẩy, ép float rồi tính điểm trung bình.

Bài 4.9: Lọc và ghi báo cáo

Chủ đề: Đọc CSV, lọc và sắp xếp

Đọc tệp diem.csv có cùng định dạng như bài 8. Ghi vào tệp baocao.txt danh sách các sinh viên có điểm trung bình từ 5 trở lên theo mẫu ho_ten diem_tb (2 chữ số thập phân), sắp xếp giảm dần theo điểm

trung bình. Trên màn hình in số sinh viên đạt và số sinh viên chưa đạt, cách nhau bởi dấu cách.

Input: Tệp diem.csv.

Output: Một dòng gồm hai số nguyên, và tệp baocao.txt.

Lưu ý: sau khi chương trình chạy xong, hệ thống sẽ tự động mở tệp `baocao.txt` và in nội dung của tệp ra để đối chiếu. Bài làm không cần tự in nội dung tệp này.

Tệp: diem.csv

```
ho_ten,toan,ly,hoa
An,8,7,9
Binh,4,4,4
Cuong,6,6,6
```

Code mẫu:

```
ket_qua = []
with open("diem.csv", "r", encoding="utf-8") as f:
    dong = f.readlines()

chua_dat = 0
for i in range(1, len(dong)):
    noi_dung = dong[i].strip()
    if noi_dung == "":
        continue
    truong = noi_dung.split(",")
    diem_tb = (float(truong[1]) + float(truong[2]) + float(truong[3])) / 3
    if diem_tb >= 5:
        ket_qua.append((diem_tb, truong[0]))
    else:
        chua_dat += 1

ket_qua.sort(reverse=True)
with open("baocao.txt", "w", encoding="utf-8") as f:
    for diem_tb, ho_ten in ket_qua:
        f.write(f"{ho_ten} {diem_tb:.2f}\n")

print(len(ket_qua), chua_dat)
```

Test case:

#	Input	Output
1	[diem.csv] ho_ten,toan,ly,hoa An,8,7,9 Binh,4,4,4 Cuong,6,6,6	2 1 --- baocao.txt --- An 8.00 Cuong 6.00
2	[diem.csv] ho_ten,toan,ly,hoa A,1,2,3	0 1 --- baocao.txt ---
3	[diem.csv] ho_ten,toan,ly,hoa X,5,5,5 Y,10,10,10	2 0 --- baocao.txt --- Y 10.00 X 5.00

Hướng dẫn: Tính trung bình, lọc từ 5 trở lên, sắp xếp giảm dần bằng list tuple rồi ghi ra tệp báo cáo.

Bài 4.10: Nhật ký nhiệt độ

Chủ đề: Đọc tệp, tuple

Tệp nhietdo.txt gồm nhiều dòng, mỗi dòng có dạng ngay nhiet_do, ví dụ 01/09 31.5. In ngày nóng nhất, ngày mát nhất và nhiệt độ trung bình (2 chữ số thập phân), mỗi kết quả trên một dòng theo mẫu NONG_NHAT ngay nhiet_do, MAT_NHAT ngay nhiet_do và TRUNG_BINH gia_tri. Nếu có nhiều ngày cùng giá trị, lấy ngày xuất hiện đầu tiên.

Input: Tệp nhietdo.txt.

Output: Ba dòng theo mẫu đã mô tả.

Tệp: nhietdo.txt

```
01/09 31.5
02/09 28.0
03/09 33.2
```

Code mẫu:

```
ban_ghi = []
with open("nhietdo.txt", "r", encoding="utf-8") as f:
    for dong in f:
        dong = dong.strip()
        if dong == "":
            continue
        ngay, nhiet_do = dong.split()
        ban_ghi.append((ngay, float(nhiet_do)))

nong_nhat = ban_ghi[0]
mat_nhat = ban_ghi[0]
tong = 0
for ngay, nhiet_do in ban_ghi:
    if nhiet_do > nong_nhat[1]:
        nong_nhat = (ngay, nhiet_do)
    if nhiet_do < mat_nhat[1]:
        mat_nhat = (ngay, nhiet_do)
    tong += nhiet_do

print("NONG_NHAT", nong_nhat[0], nong_nhat[1])
print("MAT_NHAT", mat_nhat[0], mat_nhat[1])
print(f"TRUNG_BINH {tong / len(ban_ghi):.2f}")
```

Test case:

#	Input	Output
1	[nhietdo.txt] 01/09 31.5 02/09 28.0 03/09 33.2	NONG_NHAT 03/09 33.2 MAT_NHAT 02/09 28.0 TRUNG_BINH 30.90
2	[nhietdo.txt] 10/12 18.0	NONG_NHAT 10/12 18.0 MAT_NHAT 10/12 18.0 TRUNG_BINH 18.00
3	[nhietdo.txt] 01/01 20.0 02/01 20.0 03/01 25.0	NONG_NHAT 03/01 25.0 MAT_NHAT 01/01 20.0 TRUNG_BINH 21.67

Hướng dẫn: Mỗi dòng tách thành tuple (ngày, nhiệt độ); duyệt tìm ngày nóng nhất, mát nhất và trung bình.

BÀI THỰC HÀNH SỐ 5: ĐỒ HỌA VỚI PYTHON

Tóm tắt kiến thức

Mục tiêu: Sau buổi thực hành, sinh viên vẽ được hình bằng turtle, biểu diễn dữ liệu bằng matplotlib, và trình bày kết quả trong báo cáo có trích dẫn nguồn.

1. Thư viện và cách cài đặt

turtle có sẵn trong Python; matplotlib phải cài thêm bằng `pip install matplotlib` hoặc qua giao diện PyCharm/Thonny. Chỉ cài từ kho chính thức PyPI và gõ đúng tên gói.

2. Vẽ hình với turtle

turtle mô phỏng một chú rùa cầm bút; mọi hình tạo từ lệnh di chuyển và quay.

Lệnh	Tác dụng
`t.forward(d) / t.right(g) / t.left(g)`	Đi tới / quay phải / quay trái
`t.penup() / t.pendown() / t.goto(x, y)`	Nhấc/hạ bút, di chuyển
`t.pencolor(m) / t.speed(0)`	Màu nét / tốc độ nhanh nhất
`turtle.Screen().exitonclick()`	Giữ cửa sổ tới khi bấm chuột

```
import turtle
t = turtle.Turtle()
for i in range(4):
    t.forward(100)
    t.right(90)
turtle.Screen().exitonclick()
```

Tổng các góc quay khi vẽ một hình khép kín luôn bằng 360°, nên đa giác đều n cạnh quay `360 / n` độ mỗi đỉnh. Thiếu `exitonclick()` thì cửa sổ đóng ngay và tưởng như không vẽ gì.

3. Vẽ biểu đồ với matplotlib

Một biểu đồ gồm dữ liệu, loại biểu đồ và phần chú thích (tiêu đề, nhãn trục, chú giải).

Hàm	Loại biểu đồ / tác dụng
`plt.plot(x, y, marker="o")`	Đường (dữ liệu theo thời gian)
`plt.bar(x, y) / plt.pie(gt, labels=..)`	Cột / hình quạt
`plt.scatter(x, y) / plt.hist(ds, bins=..)`	Điểm rải / tần suất
`plt.subplots(1, 2)`	Nhiều biểu đồ con
`plt.title / xlabel / ylabel / legend / grid`	Chú thích
`plt.savefig("h.png")` rồi `plt.show()`	Lưu rồi hiển thị

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [1, 4, 9], marker="o")
plt.title("Vi du")
plt.xlabel("x"); plt.ylabel("y")
plt.savefig("h.png", dpi=120)
plt.show()
```

Luôn gọi `savefig` **trước** `show`: sau khi cửa sổ đóng, hình đã bị xóa và tệp lưu ra sẽ trắng. Chọn đúng loại biểu đồ quan trọng hơn trang trí.

4. Sử dụng tài nguyên số an toàn và hợp pháp

Ghi rõ nguồn và giấy phép của dữ liệu; được tham khảo mã nguồn công khai nhưng phải ghi nguồn;

nếu dùng công cụ AI hỗ trợ phải nêu rõ và tự giải thích được toàn bộ mã của mình.

Lưu ý và lỗi thường gặp

- Thiếu `exitonclick()` nên cửa sổ turtle đóng ngay.
- Nhầm góc quay: dùng `360 / n` chứ không phải góc trong của đa giác.
- Gọi `savefig` sau `show` khiến ảnh trắng.
- Vẽ biểu đồ trực tiếp từ chuỗi đọc trong tệp mà quên ép kiểu số.
- Biểu đồ thiếu tiêu đề, nhãn trục hoặc đơn vị.
- Chạy turtle và matplotlib trong cùng một chương trình gây xung đột cửa sổ; nên tách hai tệp.

Ghi nhớ trước khi nộp bài

Lập trình trên máy, chạy thử, lưu/chụp ảnh kết quả; nộp mã nguồn (dạng văn bản) kèm ảnh và một nhận xét ngắn; kiểm tra mọi đường dẫn trong mã là đường dẫn tương đối.

Thực hành trên lớp

Bài 5.1: Đa giác đều n cạnh

Chủ đề: Đồ họa – Turtle

Nhập số nguyên n (3 ≤ n ≤ 12) từ bàn phím. Vẽ một đa giác đều n cạnh, mỗi cạnh dài 100. Góc quay tại mỗi đỉnh là 360 chia n độ.

Kết quả mong đợi: Với n = 8 vẽ ra hình bát giác đều; n càng lớn hình càng giống hình tròn.

Gợi ý:

- input() để nhập n, nhớ chuyển sang int
- Vòng lặp for chạy n lần
- left(360 / n) sau mỗi cạnh

Code mẫu:

```
import turtle

n = int(input("Nhap so canh: "))
t = turtle.Turtle()
t.speed(5)

for i in range(n):
    t.forward(100)
    t.left(360 / n)

turtle.Screen().exitonclick()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.

Hướng dẫn: Đa giác đều n cạnh có góc quay 360/n độ tại mỗi đỉnh.

Bài 5.2: Đồ thị gấp khúc

Chủ đề: Đồ họa – Matplotlib

Cho doanh thu 6 tháng: 10, 14, 9, 17, 20, 16 (triệu đồng). Vẽ biểu đồ đường thể hiện doanh thu theo tháng, có dấu tròn tại mỗi điểm, tiêu đề, nhãn hai trục và lưới. Lưu hình ra tệp.

Kết quả mong đợi: Đường gấp khúc đi qua 6 điểm, mỗi điểm có dấu tròn, có tiêu đề và nhãn trục.

Gợi ý:

- import matplotlib.pyplot as plt

- plt.plot(x, y, marker="o")
- plt.title, plt.xlabel, plt.ylabel, plt.grid(True)
- plt.savefig("ten.png") trước plt.show()

Code mẫu:

```
import matplotlib.pyplot as plt

thang = [1, 2, 3, 4, 5, 6]
doanh_thu = [10, 14, 9, 17, 20, 16]

plt.plot(thang, doanh_thu, marker="o")
plt.title("Doanh thu 6 thang dau nam")
plt.xlabel("Thang")
plt.ylabel("Doanh thu (trieu dong)")
plt.grid(True)
plt.savefig("b1.png", dpi=120)
plt.show()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.
Hướng dẫn: Vẽ đường bằng plot, thêm tiêu đề/nhãn; nhớ savefig trước show.

Bài tập tự luyện (không bắt buộc)

Bài 5.3: Các hình cơ bản

Chủ đề: Đồ họa – Turtle

Dùng thư viện turtle vẽ ba hình trên cùng một màn hình: một hình vuông cạnh 80, một tam giác đều cạnh 80, và một hình chữ nhật 120 x 60. Ba hình đặt cạnh nhau, không chồng lên nhau.
Kết quả mong đợi: Ba hình rời nhau: hình vuông, tam giác đều và hình chữ nhật.
Gợi ý:

- turtle.Turtle(), turtle.Screen()
- forward(d), right(goc), left(goc)
- penup(), pendown(), goto(x, y) để di chuyển sang vị trí mới không vẽ
- Góc quay ngoài của tam giác đều là 120 độ

Code mẫu:

```
import turtle

t = turtle.Turtle()
t.speed(3)

# Hinh vuong canh 80
for i in range(4):
    t.forward(80)
    t.right(90)

# Tam giac deu canh 80
t.penup()
t.goto(150, 0)
t.pendown()
for i in range(3):
    t.forward(80)
    t.left(120)

# Hinh chu nhat 120 x 60
t.penup()
```

```
t.goto(300, 0)
t.pendown()
for i in range(2):
    t.forward(120)
    t.right(90)
    t.forward(60)
    t.right(90)

turtle.Screen().exitonclick()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.
Hướng dẫn: Dùng penup/goto để đặt ba hình rời nhau; chú ý góc quay ngoài của đa giác.

Bài 5.4: Đường bậc thang

Chủ đề: Đồ họa – Turtle

Vẽ một đường bậc thang gồm 6 bậc. Mỗi bậc là một đoạn đi ngang 40 rồi một đoạn đi lên 40.
Kết quả mong đợi: Một đường dạng bậc thang đi từ dưới lên trên bên phải, gồm 6 bậc.
Gợi ý:

- Mỗi bậc: forward rồi quay để đi lên rồi quay lại
- Dùng một vòng lặp chạy 6 lần

Code mẫu:

```
import turtle

t = turtle.Turtle()
t.speed(5)

for i in range(6):
    t.forward(40)
    t.left(90)
    t.forward(40)
    t.right(90)

turtle.Screen().exitonclick()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.
Hướng dẫn: Lặp cặp forward–quay–forward–quay để tạo từng bậc.

Bài 5.5: Hình xoắn ốc vuông

Chủ đề: Đồ họa – Turtle

Vẽ một hình xoắn ốc vuông: bắt đầu từ một đoạn ngắn, sau mỗi lần quay trái 90 độ thì độ dài đoạn thẳng tăng thêm 5. Vẽ khoảng 40 đoạn.
Kết quả mong đợi: Một hình xoắn ốc vuông mở rộng dần ra ngoài.
Gợi ý:

- Dùng một biến do_dai, tăng dần trong vòng lặp
- forward(do_dai) rồi left(90) rồi do_dai += 5
- speed(0) để vẽ nhanh

Code mẫu:

```
import turtle

t = turtle.Turtle()
t.speed(0)
```

```
do_dai = 5
for i in range(40):
    t.forward(do_dai)
    t.left(90)
    do_dai += 5

turtle.Screen().exitonclick()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.
Hướng dẫn: Tăng dần độ dài cạnh sau mỗi lần quay để tạo xoắn ốc.

Bài 5.6: Hoa văn nhiều màu

Chủ đề: Đồ họa – Turtle

Vẽ một hoa văn gồm 36 hình vuông cạnh 100, mỗi hình xoay thêm 10 độ so với hình trước. Mỗi hình vuông tô một màu lấy lần lượt từ một danh sách màu.
Kết quả mong đợi: Một hoa văn đối xứng tâm gồm nhiều hình vuông xoay quanh một điểm, đổi màu theo chu kỳ.
Gợi ý:

- Danh sách màu, ví dụ: red, orange, gold, green, blue, purple
- pencolor(mau[i % len(mau)]) để đổi màu theo chu kỳ
- Vòng lặp ngoài 36 lần, vòng lặp trong vẽ một hình vuông, sau đó right(10)

Code mẫu:

```
import turtle

t = turtle.Turtle()
t.speed(0)
mau = ["red", "orange", "gold", "green", "blue", "purple"]

for i in range(36):
    t.pencolor(mau[i % len(mau)])
    for j in range(4):
        t.forward(100)
        t.right(90)
    t.right(10)

turtle.Screen().exitonclick()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.
Hướng dẫn: Xoay hình vuông quanh tâm và đổi màu theo chu kỳ bằng phép chia lấy dư.

Bài 5.7: Lưới ô vuông

Chủ đề: Đồ họa – Turtle

Viết hàm o_vuong(canh) vẽ một hình vuông. Dùng hàm đó và hai vòng lặp lồng nhau để vẽ một lưới 4 x 4 gồm 16 ô vuông cạnh 50, các ô đặt sát nhau.
Kết quả mong đợi: Một lưới vuông 4 hàng 4 cột gồm 16 ô vuông bằng nhau.
Gợi ý:

- Định nghĩa hàm o_vuong(canh) trước phần chính
- Hai vòng lặp lồng nhau cho hàng và cột
- penup(), goto(cot *50, hang* 50), pendown() trước khi vẽ mỗi ô

Code mẫu:

```
import turtle

t = turtle.Turtle()
t.speed(0)


def o_vuong(canh):
    for i in range(4):
        t.forward(canh)
        t.right(90)


for hang in range(4):
    for cot in range(4):
        t.penup()
        t.goto(cot * 50, hang * 50)
        t.pendown()
        o_vuong(50)

turtle.Screen().exitonclick()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.
Hướng dẫn: Viết hàm vẽ ô vuông rồi dùng hai vòng lặp lồng đặt các ô thành lưới.

Bài 5.8: Biểu đồ cột

Chủ đề: Đồ họa – Matplotlib

Cho điểm năm môn học: Toán 8.5, Lý 7.0, Hóa 9.0, Văn 6.5, Anh 8.0. Vẽ biểu đồ cột so sánh điểm các môn, trục tung giới hạn từ 0 đến 10, có tiêu đề và nhãn trục.
Kết quả mong đợi: Năm cột thể hiện điểm năm môn; cột cao nhất là Hóa, thấp nhất là Văn.
Gợi ý:

- plt.bar(ten_mon, diem, color="...")
- plt.ylim(0, 10) để cố định trục tung

Code mẫu:

```
import matplotlib.pyplot as plt

mon = ["Toan", "Ly", "Hoa", "Van", "Anh"]
diem = [8.5, 7.0, 9.0, 6.5, 8.0]

plt.bar(mon, diem, color="skyblue")
plt.title("Diem cac mon hoc")
plt.xlabel("Mon hoc")
plt.ylabel("Diem")
plt.ylim(0, 10)
plt.savefig("b2.png", dpi=120)
plt.show()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.
Hướng dẫn: Vẽ cột bằng bar, dùng ylim cố định trục tung.

Bài 5.9: Biểu đồ hình quạt

Chủ đề: Đồ họa – Matplotlib

Cho thị phần hệ điều hành di động: Android 70%, iOS 25%, Khác 5%. Vẽ biểu đồ hình quạt thể hiện tỉ lệ, hiển thị phần trăm trên mỗi phần và có tiêu đề.
Kết quả mong đợi: Biểu đồ hình quạt ba phần, phần Android chiếm phần lớn, có ghi phần trăm.

Gợi ý:

- plt.pie(gia_tri, labels=nhan, autopct="%1.1f%%")
- Tổng các giá trị nên bằng 100

Code mẫu:

```
import matplotlib.pyplot as plt

nhan = ["Android", "iOS", "Khac"]
ty_le = [70, 25, 5]

plt.pie(ty_le, labels=nhan, autopct="%1.1f%%")
plt.title("Thi phan he dieu hanh di dong")
plt.savefig("b3.png", dpi=120)
plt.show()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.

Hướng dẫn: Vẽ hình quạt bằng pie với autopct để hiện phần trăm.

Bài 5.10: Biểu đồ điểm rải

Chủ đề: Đồ họa – Matplotlib

Cho số giờ tự học và điểm thi của 10 sinh viên (dữ liệu trong code mẫu). Vẽ biểu đồ điểm rải để quan sát quan hệ giữa giờ học và điểm, có tiêu đề, nhãn trục và lưới. Viết một câu nhận xét về xu hướng.

Kết quả mong đợi: Các điểm rải cho thấy xu hướng đi lên: giờ học càng nhiều, điểm càng cao.

Gợi ý:

- plt.scatter(x, y)
- Quan sát: giờ học tăng thì điểm có xu hướng thế nào?

Code mẫu:

```
import matplotlib.pyplot as plt

gio_hoc = [1, 2, 2, 3, 4, 4, 5, 6, 7, 8]
diem = [3, 4, 5, 5, 6, 7, 6, 8, 9, 9]

plt.scatter(gio_hoc, diem, color="red")
plt.title("Quan he giua gio tu hoc va diem thi")
plt.xlabel("So gio tu hoc")
plt.ylabel("Diem thi")
plt.grid(True)
plt.savefig("b4.png", dpi=120)
plt.show()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.

Hướng dẫn: Vẽ điểm rải bằng scatter để quan sát quan hệ hai đại lượng.

Bài 5.11: Đồ thị hàm số

Chủ đề: Đồ họa – Matplotlib

Vẽ đồ thị hai hàm số y = x mũ 2 và y = x mũ 3 trên cùng một hệ trục, với x chạy từ -10 đến 10 (bước nhảy 1). Có chú giải phân biệt hai đường, tiêu đề, nhãn trục và lưới. Tự sinh danh sách x và y bằng vòng lặp.

Kết quả mong đợi: Hai đường cong: parabol y = x^2 và đường bậc ba y = x^3, có chú giải.

Gợi ý:

- Dùng vòng lặp while hoặc for để tạo danh sách x, y2, y3

- plt.plot(x, y, label="...") rồi plt.legend()

Code mẫu:

```
import matplotlib.pyplot as plt

x = []
y2 = []
y3 = []
gia_tri = -10
while gia_tri <= 10:
    x.append(gia_tri)
    y2.append(gia_tri ** 2)
    y3.append(gia_tri ** 3)
    gia_tri += 1

plt.plot(x, y2, label="y = x^2")
plt.plot(x, y3, label="y = x^3")
plt.title("Do thi ham so bac hai va bac ba")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.savefig("b5.png", dpi=120)
plt.show()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.

Hướng dẫn: Tự sinh danh sách x, y bằng vòng lặp rồi vẽ hai đường kèm legend.

Bài 5.12: Đồ thị sin và cos

Chủ đề: Đồ họa – Matplotlib

Vẽ đồ thị hai hàm sin và cos theo góc từ 0 đến 360 độ (bước nhảy 5 độ) trên cùng một hệ trục, có chú giải, tiêu đề, nhãn trục và lưới. Nhớ đổi độ sang radian trước khi tính.

Kết quả mong đợi: Hai đường hình sóng sin và cos lệch pha nhau, dao động trong khoảng -1 đến 1.

Gợi ý:

- import math và dùng math.radians, math.sin, math.cos
- Tạo danh sách góc, danh sách sin, danh sách cos bằng vòng lặp
- plt.legend() để hiện chú giải

Code mẫu:

```
import matplotlib.pyplot as plt
import math

x = []
gia_tri_sin = []
gia_tri_cos = []
goc = 0
while goc <= 360:
    radian = math.radians(goc)
    x.append(goc)
    gia_tri_sin.append(math.sin(radian))
    gia_tri_cos.append(math.cos(radian))
    goc += 5

plt.plot(x, gia_tri_sin, label="sin")
plt.plot(x, gia_tri_cos, label="cos")
plt.title("Do thi ham sin va cos")
plt.xlabel("Goc (do)")
```

```
plt.ylabel("Gia tri")
plt.legend()
plt.grid(True)
plt.savefig("b6.png", dpi=120)
plt.show()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.

Hướng dẫn: Dùng math.radians đổi độ sang radian rồi vẽ hai đường sóng sin, cos.

Bài 5.13: Biểu đồ tần suất

Chủ đề: Đồ họa – Matplotlib

Cho danh sách điểm của 20 sinh viên (trong code mẫu). Vẽ biểu đồ tần suất (histogram) với các khoảng 0-2, 2-4, 4-6, 6-8, 8-10 để xem phân bố điểm, có tiêu đề, nhãn trục và viền cột.

Kết quả mong đợi: Biểu đồ tần suất cho thấy phần lớn sinh viên đạt điểm trong khoảng 6 đến 8.

Gợi ý:

- plt.hist(dulieu, bins=[0, 2, 4, 6, 8, 10], edgecolor="black")
- bins là danh sách các mốc chia khoảng

Code mẫu:

```
import matplotlib.pyplot as plt

diem = [5, 6, 6, 7, 7, 7, 8, 8, 9, 10, 4, 6, 7, 8, 5, 9, 7, 6, 8, 7]

plt.hist(diem, bins=[0, 2, 4, 6, 8, 10], color="green", edgecolor="black")
plt.title("Phan bo diem cua lop")
plt.xlabel("Khoang diem")
plt.ylabel("So sinh vien")
plt.savefig("b7.png", dpi=120)
plt.show()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.

Hướng dẫn: Vẽ biểu đồ tần suất bằng hist với danh sách bins chia khoảng.

Bài 5.14: Kết hợp nhiều biểu đồ con

Chủ đề: Đồ họa – Matplotlib

Trong một cửa sổ, vẽ hai biểu đồ nằm cạnh nhau: bên trái là biểu đồ đường doanh thu 5 tháng, bên phải là biểu đồ cột số lượng ba sản phẩm. Mỗi biểu đồ có tiêu đề riêng.

Kết quả mong đợi: Cửa sổ chứa hai biểu đồ cạnh nhau: một đường và một cột, mỗi biểu đồ có tiêu đề riêng.

Gợi ý:

- hinh, truc = plt.subplots(1, 2, figsize=(10, 4))
- truc[0].plot(...), truc[0].set_title(...)
- truc[1].bar(...), truc[1].set_title(...)

Code mẫu:

```
import matplotlib.pyplot as plt

thang = [1, 2, 3, 4, 5]
doanh_thu = [10, 12, 9, 15, 14]
san_pham = ["A", "B", "C"]
so_luong = [5, 9, 4]
```

```
hinh, truc = plt.subplots(1, 2, figsize=(10, 4))
truc[0].plot(thang, doanh_thu, marker="o")
truc[0].set_title("Doanh thu theo thang")
truc[0].set_xlabel("Thang")
truc[0].set_ylabel("Trieu dong")
truc[1].bar(san_pham, so_luong, color="orange")
truc[1].set_title("So luong san pham")
truc[1].set_xlabel("San pham")
truc[1].set_ylabel("So luong")
plt.tight_layout()
plt.savefig("b8.png", dpi=120)
plt.show()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.
Hướng dẫn: Dùng plt.subplots đặt hai biểu đồ cạnh nhau trong một hình.

Bài 5.15: Đường cong tham số

Chủ đề: Đồ họa – Matplotlib

Dùng tọa độ tham số theo góc t từ 0 đến 360 độ để vẽ hai hình cạnh nhau: bên trái là một đường tròn bán kính 1 (x = cos t, y = sin t), bên phải là một đường hoa hồng tám cánh với bán kính r = cos(4t), rồi x = r cos t, y = r sin t. Đặt tỉ lệ trục bằng nhau để hình không bị méo.
Kết quả mong đợi: Bên trái một đường tròn, bên phải một đường hoa hồng tám cánh đối xứng.
Gợi ý:

- import math, tạo danh sách điểm bằng vòng lặp qua t
- set_aspect("equal") để hai trục cùng tỉ lệ
- Đường hoa hồng: r = cos(4 * t) rồi nhân với cos t và sin t

Code mẫu:

```
import matplotlib.pyplot as plt
import math

x_tron = []
y_tron = []
x_hoa = []
y_hoa = []
goc = 0
while goc <= 360:
    t = math.radians(goc)
    x_tron.append(math.cos(t))
    y_tron.append(math.sin(t))
    r = math.cos(4 * t)
    x_hoa.append(r * math.cos(t))
    y_hoa.append(r * math.sin(t))
    goc += 1

hinh, truc = plt.subplots(1, 2, figsize=(10, 5))
truc[0].plot(x_tron, y_tron)
truc[0].set_title("Duong tron")
truc[0].set_aspect("equal")
truc[1].plot(x_hoa, y_hoa, color="purple")
truc[1].set_title("Duong hoa hong r = cos(4t)")
truc[1].set_aspect("equal")
plt.savefig("b9.png", dpi=120)
plt.show()
```

Chấm: sinh viên nộp mã nguồn và ảnh kết quả; đối chiếu với mô tả kết quả mong đợi.

Hướng dẫn: Dùng tọa độ tham số theo góc và set_aspect('equal') để hình không méo.